ELEC 490/498 Final Report

Sustainable Supercomputing Using Power Controls to Maximize Performance and Minimize Energy Usage

Group #: 30

Group members:

1) Johnnie Tse (22yht, 20366054)

2) Gia Lee (19jl253, 20231785)

3) Zane Prance (20zdtp, 20233463)

4) Valerie So (20wyvs, 20291603)

Faculty supervisor: Dr. Ryan Grant

Submission date: April 7th, 2026

Executive Summary

High-performance computing (HPC) systems, increasingly driven by AI workloads, consume massive amounts of energy and water, raising environmental and economic concerns. While it is important to address these problems, it is also important to work with involved stakeholders such as the Centre for Advanced Computing, which prohibits system-level changes to maintain shared resources, user privacy, and fair resource allocation. To navigate these constraints, a user-level, phase-aware Dynamic Voltage and Frequency Scaling (DVFS) approach was developed to reduce energy usage without compromising system stability or requiring privileged access.

To realize this goal, the project designed and implemented a phase-aware DVFS controller for miniMD built from three subsystems: modification to miniMD's source code to publish per-rank phase hints, an external Python monitor that safely reads the application hints and applies per-core DVFS policies, and a bash runtime script that handles application launch, core placement and cleanup. This architecture enables low-overhead per-core frequency control using standard Linux interfaces, while keeping compute phases at full performance and reducing frequency only during less performance-critical phases.

To test and evaluate the solution, testing followed the Blueprint's statistical framework using 25 repetitions per configuration, RAPL energy measurements, and Welch's t-tests ($p < 0.05$). The controller was tested on six MPI rank configurations ($N = 1$ to 26) on the dedicated Frontenac node. All functional and interface specifications were met, phase detection accuracy reached approximately 100% through application hints, frequency scaling latency averaged 0.31 ms, and application stability was perfect over 300 runs. Energy savings reached 5.70% at $N = 16$ and 17.53% at $N = 26$, with runtime overhead below 3.21% (well within the ≤8% target).

The results showed that 11 of 12 required Blueprint specifications were fully or substantially met, and 2 of 7 optional enhancements were achieved, including a live dashboard, and >10% energy saving. The energy saving target was partially met overall (average 3.09%) but fully met at operational scales ($N \geq 8$, average 8.03%). The two caveated specifications (phase detection and control loop overhead) were justified by evidence-driven design pivots allowed in the Blueprint. No specification was completely unmet.

Altogether, the final implementation shows that energy savings are achievable on shared supercomputing systems using only user-accessible per-core frequency controls. These

findings suggest that phase-aware power management is an effective approach for improving HPC energy efficiency in restricted environments. Future improvements can include the implementation of uncore or bus frequency to determine if additional savings are possible.

Table of Contents

Table of Tables

Table of Figures

1. Introduction

High-performance computing (HPC) systems play a critical role in advancing scientific research, enabling large-scale simulations in fields such as molecular dynamics, climate modeling, and engineering analysis. Recent growth in AI, particularly the training of large-scale models, has significantly increased demand for HPC infrastructure [1]. However, this growth has resulted in a growing environmental impact via its energy consumption and water usage for its cooling infrastructure [2]. In many cases, HPC systems consume energy on the scale of small cities, creating both economic and environmental challenges [3]. The Frontier supercomputer at Oak Ridge National Laboratory draws 22.7 MW, equivalent to powering approximately 18,000 residential homes [4]. The International Energy Agency reports that global data center electricity consumption reached 460 TWh in 2024, with HPC workloads comprising approximately 15% of this figure [5]. Within Canada, the Centre for Advanced Computing at Queen's University operates the Frontenac cluster as a shared national resource, where energy efficiency improvements benefit the entire Canadian research community. From an ethical and cost standpoint, reducing HPC energy consumption directly contributes to lowering the carbon footprint of computational research and reducing the operational expenses faced by HPC facilities.

A key source of energy inefficiency in HPC systems is that processors typically operate at maximum frequency regardless of workload demands [6]. Applications consist of different phases, including compute-intensive, communication, memory-bound, and input/output (I/O) operations. While high CPU frequency is necessary during compute-heavy phases to

maintain performance, it is not necessary during communication or I/O phases, when the processor is idle waiting on data transfer or synchronization. To study and optimize phase-specific power inefficiencies, miniMD was the chosen application. miniMD is a proxy application for molecular dynamics simulations, well-suited for this study since it includes communication and I/O phases, which will be optimized.

Dynamic Voltage and Frequency Scaling (DVFS) is a widely used technique that reduces processor power consumption by lowering the voltage and clock frequency when full performance is not needed. DVFS utilizes hardware performance counters to detect workload phases and adjusts frequency accordingly [6].

Opposed to traditional DVFS, a phase-aware DVFS controller was implemented for the miniMD molecular dynamics proxy application. The Centre for Advanced Computing (CAC) provided access to the supercomputing infrastructure and, as a primary stakeholder, needs to maintain system stability and fair resource sharing across all users. This influenced the solution such that the controller was built entirely within user-level permissions, avoiding any low-level hardware interference that could affect other users or system integrity. This constraint also informed safety considerations, as any solution that risked destabilizing shared infrastructure or causing unintended interactions between concurrent jobs was ruled out. In phase-aware DVFS, the application directly communicates its current workload phase to the controller via shared memory, allowing for precise and responsive frequency scaling decisions. Furthermore, power was adjusted

only via CPU scaling, as it was the only mechanism available through user-level permission. With respect to privacy, the controller operates only on local performance and phase data generated by the application itself, with no collection or transmission of user data, raising no privacy concerns.

This report makes the following contributions to the field of energy-efficient high-performance computing:

1. A user-level, phase-aware DVFS architecture that achieves up to 17.53% energy savings on shared HPC infrastructure without requiring privileged system access, demonstrating that meaningful energy reductions are achievable within the operational constraints of multi-tenant supercomputing facilities.
2. An empirical comparison of three controller architectures (counter-based inference, adaptive beta algorithm, and application-injected hints), providing evidence-driven justification for the superiority of explicit phase communication over indirect inference methods on AMD Zen 1 microarchitectures.
3. A statistically rigorous experimental evaluation spanning 300 runs across 6 MPI rank configurations, with Welch's t-tests ($p < 0.001$ for all configurations) and Cohen's d effect sizes exceeding 15 standard deviations at operational scale, establishing the controller's effectiveness with high statistical confidence.
4. An open-source, deployable implementation consisting of approximately 500 lines of code (150 C++, 250 Python, 100 bash) that can be adopted on any Linux system

with the standard cpufreq interface, lowering the barrier to entry for phase-aware power management in academic HPC environments.

2. Design

The objective of the project was to develop and evaluate a phase-aware Dynamic Voltage and Frequency Scaling (DVFS) controller for the application miniMD that reduces CPU energy consumption while maintaining runtime performance. The central design problem was not just simply how to reduce processor frequency, but instead how to determine when such a reduction would be beneficial or harmful to the application. In an HPC workload, some phases of execution are compute-heavy while others are dominated by rank communication, synchronization, or I/O operations and may tolerate lower CPU operating points with minimal runtime impact. Thus, the design target was a closed-loop runtime controller that could observe application behavior, infer the appropriate control response, and apply DVFS decisions with low enough overhead such that the controller did not distort the workload it was optimizing.

The system had to be implemented using the interfaces available on the target Linux platform, had to operate alongside an MPI-based workload without destabilizing the execution, as well as finding a balance between energy reduction and runtime performance. The result was a phase-aware architecture in which the application is responsible for publishing its current phase for each MPI rank, an external monitor that

interprets the phase information, and a lightweight control path that adjusts per-core CPU frequency accordingly.

2.1 Functional Requirements and Constraints

Several functional requirements governed the design. First, the system had to expose the current execution state of the application in a form that could be consumed externally. Second, it had to support per-core control decisions rather than coarse system-wide policies, since different ranks may not always exhibit identical behavior. Third, it had to preserve performance during compute-bound phases by avoiding unnecessary downclocking when the CPU was required to be operating at max capacity. Fourth, it had to remain lightweight such that the control system itself did not become a source of runtime overhead. Lastly, it had to run entirely in software using the standard Linux interfaces, without any kernel-level modifications, as the team was only provided basic user space permissions. These requirements shaped the project into three primary subsystems: application-side phase publication, external monitoring and policy enforcement, and runtime orchestration.

These constraints helped shape the design philosophy of the final system. The controller had to operate on a Linux machine using the available ‘cpufreq’ ‘sysfs’ interface, meaning that the policy updates had to be implemented through governor and set-speed writes rather than custom kernel mechanisms. The benchmark was an MPI application, miniMD, distributed across multiple worker cores, so the monitor had to avoid interfering with the

application's execution. In addition, the communication pathway connecting the application and the controller needed to remain lightweight, since excessive synchronization overhead would undermine the value of the design. These constraints motivated a software architecture based on explicit shared-memory managed hints, a separate Python monitor running on its own core, and a simple policy engine that maps application phases to operating modes. The script configuration shows this separation clearly, with the monitor being exclusively pinned to core 30 and made to ignore core 31, as it was reserved for the node.

Table 1: Project Requirements and Responsive Action

Requirement / Constraint	Design Response
Per-rank phase visibility	Added shared-memory phase table to miniMD
The external controller must interpret the runtime state	Implemented a Python monitor using the same shared layout
Compute-intensive execution must retain performance	Compute phases always map to performance mode
Overhead must remain low	Lock-free sequence protocol and dedicated monitor core
Must operate within available Linux interfaces	Used ‘cpufreq’ and ‘/dev/shm’ shared memory

2.2 DVFS

DVFS was selected because the processor power depends strongly on operating voltage and clock frequency. An approximation of the dynamic power consumed by a processor is:

$$P_{dyn} = \alpha C V^2 f$$

where α is the activity factor, C is the effective switched capacitance, V is the supply voltage, and f is the clock frequency. Total processor power can then be expressed as follows:

$$P_{total} = P_{dyn} + P_{static}$$

and the total energy consumed over an execution interval is:

$$E = \int P(t)dt$$

These relationships justify why DVFS can serve as a tool for energy reduction because lowering voltage and frequency can reduce processor power. Given the limited permissions the team had, we could not directly change the voltage consumed by the processor, but we could adjust the clock frequency, and thus this became the primary parameter the system would target. In HPC workloads, lowering the frequency during compute-critical regions can increase the runtime enough to offset any power savings. During heavy communication or I/O heavy phases, in contrast, the CPU may spend most of its time waiting for or transferring data, which makes these phases better candidates for reduced frequency operation since they do not depend on instruction throughput. The effectiveness of the DVFS, therefore, depends on whether the processor is on the critical path during a given phase. For this reason, the project adopted a phase-aware design rather than a static frequency policy. A simple phase-level energy model can be written as:

$$E = P_c T_c + P_m T_m + P_i T_i$$

where P_c, P_m, and P_i, are the average powers during compute, communication, and I/O phases, and T_c, T_m, T_i are the corresponding phase durations. In compute-intensive phases, reducing frequency may substantially increase T_c, which can harm both

performance and increase energy consumed. In communication-heavy or I/O phases, the CPU is often not the limiting resource; thus, reducing frequency can lower power with a much smaller performance impact. This is the core design rationale for the final system: compute phases should remain at maximum performance, while clearly lower-value phases should be eligible for downclocking. The CoPPer paper reinforces this design logic as it notes that DVFS has historically been the cornerstone of software approaches for meeting performance goals with minimal energy [7].

This logic is reflected directly in the controller implementation. The monitor maps ‘PHASE_COMPUTE’ and ‘PHASE_SYNTH_ACTIVE’ to immediate performance mode, while ‘PHASE_IO’ and ‘PHASE_SYNTH_WAIT’ transition to a lower-frequency mode only after they persist for at least ‘low_after_ms’. Similarly, the regular internal MPI phases ‘PHASE_COMMUNICATE’, ‘PHASE_EXCHANGE’, ‘PHASE_BORDERS’, and ‘PHASE_REVERSE’ transition only to the medium-frequency operation mode after persisting for at least ‘mid_after_ms’. The configured reduced frequency modes are 1.2 GHZ and 1.6GHZ, with default persistence thresholds of 2ms and 5ms, respectively.

2.3 Engineering Path to Final Design

The final hint-based approach was not the first design solution explored. The design process for this project was iterative and evidence-driven. This methodology was adopted from The Case of the Missing Supercomputer Performance, which argued that HPC systems are by their very nature complex and should be approached through disciplined

comparison of expected and observed behavior, followed by hypothesis testing and design refinement.

The first design attempted to detect execution phases indirectly using hardware performance counters. The advantage of this approach was that it would not require any modification of the original miniMD code. In theory, metrics like IPC, cache behavior, and stalls could have been correlated to high-level phases and used to drive the controller. However, early validation testing showed this approach had only achieved about 3% phase-detection accuracy, which made it unsuitable for a reliable closed-loop system. After this approach failed, a second intermediate design was explored based on the beta adaptation algorithm [8]. This work proposes an interval-based runtime DVFS controller that estimates a beta parameter describing how CPU-bound the current execution interval is, then computes the lowest frequency that should satisfy the user-chosen performance slowdown bound gamma. The algorithm operates by measuring millions of instructions per second (MIPS) at available frequencies, estimating beta through regression, and using that to choose a target operating frequency. When the desired operating point falls between supported operating frequencies, the controller can emulate the target frequency by running the neighboring frequencies in a specific ratio. A prototype using this idea was implemented for the project. It sampled per-core utilization from '/proc/state', measured write intensity, and gathered metrics from 'perf stat'. It maintained per-core beta estimates from observed MIPS behavior. It also supported the emulation of intermediary frequencies. This prototype was good as it went beyond thresholding and attempted to estimate

workload sensitivity with DVFS directly. However, in practice, it increased the runtime severely. The repeated process scanning, perf-based IPC monitoring, beta updates, and frequency writes introduced too much monitoring overhead. In addition, the per-core nature of the emulated frequency increased the latency in rank-to-rank communication, which was the portion of the execution most responsible for the increased runtime. Therefore, instead of behaving like a lightweight controller, the prototype became a measurable source of interference.

This intermediate design stage was still very valuable because it clarified what the final system would need to avoid. It demonstrated that even a theoretically well-motivated adaptive controller can still be a poor engineering choice if its overhead is too high relative to the workload. This led to the final design pivot, which was the adoption of explicit application-generated phase hints. By moving phase classification over to be the responsibility of the application itself, the design eliminated the need for expensive external phase inferencing and produced a more reliable and lower-overhead solution for control. This final version offered the best balance of observability, control quality, and runtime cost.

2.3.1 Formal Design Alternative Evaluation

To systematically justify the final design selection, a weighted Pugh decision matrix was constructed, evaluating the three candidate architectures against five engineering criteria.

Each criterion was assigned a weight reflecting its importance to the project's success, informed by the Blueprint's specification priorities and the CAC's operational constraints.

Table 2: Weighted Decision Matrix for Controller Architecture Selection

Criterion	Weight	Counter-Based Detection	Beta Algorithm	Hint-Based (Final)
Phase Detection Accuracy	0.30	1 (3% measured)	3 (indirect, ~40% est.)	5 (~100% measured)
Runtime Overhead	0.25	4 (low: read-only counters)	1 (97.2% at N=2)	4 (<3.21% worst case)
Implementation Complexity	0.15	2 (counter correlation tuning)	1 (PID + beta regression + freq emulation)	5 (~500 LOC total)
Hardware Portability	0.15	1 (Zen 1-specific IPC signatures)	2 (requires /proc/stat + perf)	4 (any Linux with cpufreq)
Maintainability	0.15	2 (fragile thresholds)	1 (complex state machine)	5 (simple phase-to-policy map)
Weighted Score		2.15	1.70	4.65

Scoring: 1 = Poor, 2 = Below Average, 3 = Average, 4 = Good, 5 = Excellent.

The hint-based architecture scored 4.65/5.00 m, which is more than double the Beta algorithm (1.70) and the counter-based approach (2.15). The dominant factor was detection accuracy (weight 0.30), where the hint-based approach's ~100% accuracy was decisive. The Beta algorithm scored lowest overall due to its catastrophic runtime overhead, which negated any potential energy benefit.

2.3.2 Trade-Space Visualization

The design trade-space can be visualized by plotting the three alternatives on axes of Detection Accuracy versus Runtime Overhead (see Figure 18). An ideal controller occupies the upper-left quadrant (high accuracy, low overhead). The counter-based approach achieves low overhead but fails in accuracy. The Beta algorithm fails on both dimensions.

Only the hint-based approach occupies the optimal region of the trade-space, confirming it as the Pareto-dominant design.

2.4 Final System Architecture

The final architecture consisted of three main subsystems. The first was the modified miniMD application, specifically integrate.cpp, which was extended to publish the active phase of each MPI rank into a shared memory table. The second was an external Python monitor that attached to that same table, read stable snapshots of the rank states, and translated those states into per-core-based DVFS actions, using the Linux 'cpufreq' interface. The third subsystem was a bash-based runtime infrastructure that launches the application and the controller with the required environment variables, core placement, and cleanup behavior. These three components form the complete implemented system. This architecture was chosen as it has separate responsibilities cleanly. The application is responsible for publishing the phase information. The monitor is responsible only for interpreting that information and applying the control policy. The runtime script is responsible only for orchestration, system setup, and cleanup. This separation improved modularity and made the project easier to debug and develop. Figure 1 below shows the conceptual structure of the system.

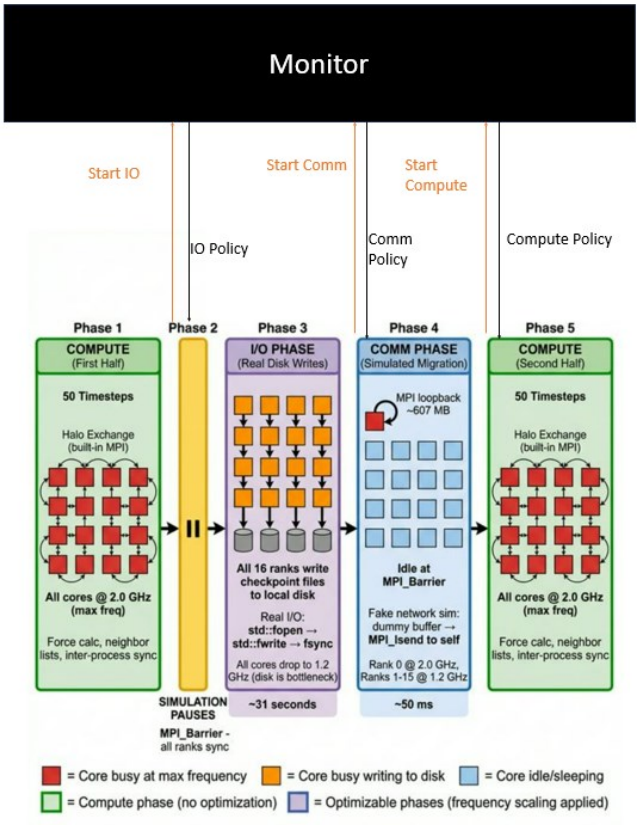

Figure 1: High-level overview of the system's structure

2.5 Key Decisions Trade-offs

One major decision was to use shared memory hints instead of relying on hardware counter inference. Shared memory provided a lightweight mechanism for communication between the application and the controller. Another important design decision was to avoid reacting immediately to every phase transition. Instead, the controller applies threshold-based decisions so that very short transient phases do not trigger unnecessary frequency changes. This prevented the controller from overreacting to very short transient phases where the cost of the policy switch may outweigh the benefit.

Another trade-off involved core allocation. Because the monitor is itself a software process, it could interfere with the cores executing miniMD if allowed to compute for CPU

time with the worker ranks. To prevent this, the runtime script pins the monitor to core 30 and leaves core 31 reserved. This design provides a physical separation between the application and controller, removing the chance that the monitoring activity will directly affect the CPU time for the workload. This helps to keep the control mechanism lightweight.

3. Implementation

Following the design revision discussed in Section 2, the final system was implemented as a hint-based phase-aware DVFS controller for miniMD. Rather than attempting to infer phase from the hardware performance counters, the completed design used explicit phase hints generated inside the application to expose the current execution state of each MPI rank to the external controller. The final implementation consisted of three tightly coupled components: modifications to integrate.cpp to publish phase information, a Python monitor that interpreted those hints and adjusted CPU frequency policies, and a bash runtime script that launched the application and controller with a consistent configuration.

3.1 miniMD Phase Hint Instrumentation

The first major implementation task was to modify the primary runtime loop in the file integrate.cpp so that miniMD could communicate its execution phases to an external controller with minimal overhead. To accomplish this, a shared memory phase table was added directly into the application. The code defines a ‘PhaseCode’ enumeration for the major execution phases, a ‘PhaseSlot’ structure that contains the sequence number, rank

core, phase, and timestamp for one MPI rank, and a ‘PhaseTable’ structure containing the table header and an array of slots. This gives both the application and the controller a common memory layout for exchanging phase information. The table is identified by a fixed magic value, ‘PHASE_MAGIC’, and supports up to 64 slots.

```
enum PhaseCode : uint32_t {
  PHASE_COMPUTE      = 0,
  PHASE_COMMUNICATE  = 1,
  PHASE_EXCHANGE     = 2,
  PHASE_BORDERS      = 3,
  PHASE_REVERSE      = 4,
  PHASE_IO           = 5,
  PHASE_SYNTH_ACTIVE = 6,
  PHASE_SYNTH_WAIT   = 7,
  PHASE_DONE         = 8
};

struct PhaseSlot {
  volatile uint32_t seq;
  volatile int32_t rank;
  volatile int32_t core;
  volatile uint32_t phase;
  volatile uint64_t t_ns;
};

static const uint32_t PHASE_MAGIC = 0x50485331u; // "PHS1"
static const int MAX_PHASE_SLOTS = 64;

struct PhaseTable {
  uint32_t magic;
  uint32_t nslots;
  PhaseSlot slots[MAX_PHASE_SLOTS];
};
```

Figure 2: Shared-memory data structure for inter-process communication. The PhaseTable and PhaseSlot structs provide the fixed-memory layout required for the instrumented miniMD ranks to asynchronously publish execution hints to the external frequency controller.

The initialization of the hinting mechanism begins as rank 0 creates the shared memory file. It then maps the file, clears the table, and populates the header fields of the table, including the magic number and slot count. After an ‘MPI_barrier’, each rank opens the shared file, memory-maps it, stores its rank index, and publishes an initial ‘PHASE_COMPUTE’ state. When miniMD’s execution completes, the cleanup routine is invoked and writes ‘PHASE_DONE’ to each slot, synchronizes the ranks, unmaps the table,

closes the file descriptor, and removes the shared-memory file. This ensures that each run begins from a clean slate and that the monitor will always attach to the latest table rather than stale data from a previous run.

Phase updates were written through a dedicated ‘phase_hint_write()’ function. Ranks invoke this function to update their slot in the table. They begin by incrementing the sequence number, so it is an odd value to mark a write in progress, then store the rank, current CPU core, phase code, and timestamp, and finally increment the sequence value again to an even number to signal the snapshot was stable. The use of even and odd sequence numbers allowed the monitor to distinguish between partial and fully written updates without the use of locks. The result is a lightweight communication mechanism that performs a single 8-byte write per transition and remains safe for concurrent observation by the external controller.

```
static void phase_hint_write(uint32_t phase) {
  if (!g_phase_table || g_phase_rank < 0 || g_phase_rank >= MAX_PHASE_SLOTS)
    return;

  PhaseSlot &s = g_phase_table->slots[g_phase_rank];

  uint32_t seq = s.seq + 1; // odd = write in progress
  s.seq = seq;
  __sync_synchronize();

  s.rank = g_phase_rank;
  s.core = sched_getcpu();
  s.phase = phase;
  s.t_ns = monotonic_ns();

  __sync_synchronize();
  s.seq = seq + 1; // even = stable snapshot
}
```

Figure 3: Method to update the core state from the application

These phase writes were then inserted at meaningful boundaries in the simulation loop. ‘PHASE_COMMUNICATE’ is written before explicit MPI communication, ‘PHASE_EXCHANGE’ and ‘PHASE_BORDER’ are written before the corresponding particle movement stages, and ‘PHASE_REVERSE’ is written before reverse communication. After these non-compute regions are complete, the application explicitly restores its state to ‘PHASE_COMPUTE’. This gives the controller direct visibility into the transition between compute-dominated and communication-dominated behavior.

In addition to the phase publication, the application was also extended with checkpoint and synthetic network support. At the midpoint of the run, miniMD publishes ‘PHASE_IO’ and invokes a sustained checkpoint method. This function starts with rank 0, preparing a checkpoint directory. Once it is created, every rank then writes its own checkpoint file containing a header with simulation data such as positions, velocities, and forces. Rather than performing a short write and immediately returning, the function was configured to write the data in chunks and, if required, issue padded writes until the I/O duration was reached, which for this project was set to 30 seconds. This was done to simulate a storage-heavy region rather than a quick burst, making the I/O phase long enough for the external controller to detect and respond to.

miniMD then enters a synthetic communication phase that was designed to emulate a prolonged network communication-dominated runtime. During this phase, rank 0 publishes ‘PHASE_SYNTH_ACTIVE’, while the remaining worker ranks publish

‘PHASE_SYNTH_WAIT’. This allows the external controller to distinguish between ranks that are actively engaging in the simulated network and those that are waiting on it. The communication function uses the volume of data present in the simulation and then performs repeated MPI loopback send and receive operations in chunks until the target communication time is reached. If the data is exhausted before the target time passes, the function continues issuing padding traffic so that the phase remains active for the whole interval. After this synthetic phase finishes, miniMD returns all ranks to ‘PHASE_COMPUTE’ and resumes the second half of the simulation.

These additions broadened the set of phases exposed by miniMD and made the implemented system more representative of mixed HPC behavior.

3.2 Python Phase Monitor and DVFS Controller

The second major component was the external Python monitor, implemented in comm_freq_controller.py. This program replicated the same shared-memory layout used in miniMD by defining matching ‘PhaseSlot’ and ‘PhaseTable’ structures using ‘ctypes’. At startup, the monitor waits until the hint file exists, memory-maps the shared table, and then waits until the expected magic value and nonzero slot count are present. This allows the controller to launch independently of the benchmark process while still attaching safely once the application has created and initialized the hint table.

To read hints safely, the monitor used a ‘snapshot_slot()’ function. This function checks the slot sequence value before and after reading the slot contents. If the sequence number is odd, the monitor treats the slot as currently being updated, and the monitor retries. If the sequence value is unchanged and even, the snapshot is accepted as stable. This directly matches the write protocol used inside the integrate.cpp file in miniMD and allows the controller to observe rank state without lock contention or inconsistent reads.

```
def snapshot_slot(slot):
    while True:
        seq1 = slot.seq
        if seq1 & 1:
            continue
        rank = slot.rank
        core = slot.core
        phase = slot.phase
        t_ns = slot.t_ns
        seq2 = slot.seq
        if seq1 == seq2 and not (seq2 & 1):
            return rank, core, phase, t_ns
```

Figure 4: Routine monitor is used to access application hints

The control logic itself was implemented in ‘desired_policy()’. Compute heavy phase, including ‘Phase_Compute’, and ‘Phase_SYNTH_ACTIVE’, always mapped immediately to ‘performance’ mode so that the critical work being done on the core would not be slowed down unnecessarily. Longer, lower-value phases such as ‘PHASE_IO’ and ‘PHASE_SYNTH_WAIT’ only trigger downclocking after persisting for at least ‘low_after_ms’, while regular internal communication phases such as ‘PHASE_COMMUNICATE’, ‘PHASE_EXCHANGE’, ‘PHASE_BORDERS’, PHASE_REVERSE’ only trigger a smaller frequency reduction after persisting for at least ‘mid_after_ms’. The consistent thresholds

prevent the controller from reacting to sub-millisecond phase transitions, where the energy cost of a frequency transition (PLL re-lock at 0.1 to 0.3 ms) would exceed the energy saved during the brief phase. This creates a direct phase-to-policy mapping that reflects the design rationale described in Section 2 above. In the final configuration, the low-frequency mode is 1.2GHz, and the medium-frequency mode is 1.6GHz, with default thresholds of 2 ms and 5 ms, respectively (please see Table 12 in the Appendix).

```
def desired_policy(phase, age_ms, args):
    # Always restore compute-like phases immediately.
    if phase in (PHASE_COMPUTE, PHASE_SYNTH_ACTIVE, PHASE_DONE):
        return ("performance", None)

    # Long clearly low-value phases.
    if phase in (PHASE_IO, PHASE_SYNTH_WAIT):
        if age_ms >= args.low_after_ms:
            return ("userspace", args.freq_low)
        return ("performance", None)

    # Regular internal MPI phases (only touch them if they persis)
    if phase in (PHASE_COMMUNICATE, PHASE_EXCHANGE, PHASE_BORDERS, PHASE_REVERSE):
        if age_ms >= args.mid_after_ms:
            return ("userspace", args.freq_mid)
        return ("performance", None)

    return ("performance", None)
```

Figure 5: Control logic used by the monitor

Once the desired policy had been selected, the controller applied it on a per-core basis through Linux's 'cpufreq sysfs' interface. The script writes 'performance' to the 'scaling_governor' when full speed is requested and otherwise switches to the 'userspace' governor before writing the requested frequency value to 'scaling_setspeed'. To reduce unnecessary write times, the monitor stores the last applied mode and frequency for each core in a cache and skips redundant updates. It also ignores the reserved core and the monitor's own core, preventing the controller from interfering with itself or wasting cycles

trying to interact with a dedicated reserved CPU core. During shutdown, the monitor restores all observed worker cores to ‘performance’ mode. These safeguards helped ensure that the controller remained lightweight and did not leave the system in an unexpected state after an experiment ended.

```
def apply_mode(core, mode, freq, cache, dry_run=False):
    prev = cache.get(core)
    desired = (mode, freq)
    if prev == desired:
        return

    if mode == "performance":
        set_governor(core, "performance", dry_run)
    elif mode == "userspace":
        set_governor(core, "userspace", dry_run)
        set_freq(core, freq, dry_run)

    cache[core] = desired
```

Figure 6: A routine that applies the selected policy to the core

Figure 7 below illustrates the CPU frequency scaling policy as a state machine diagram.

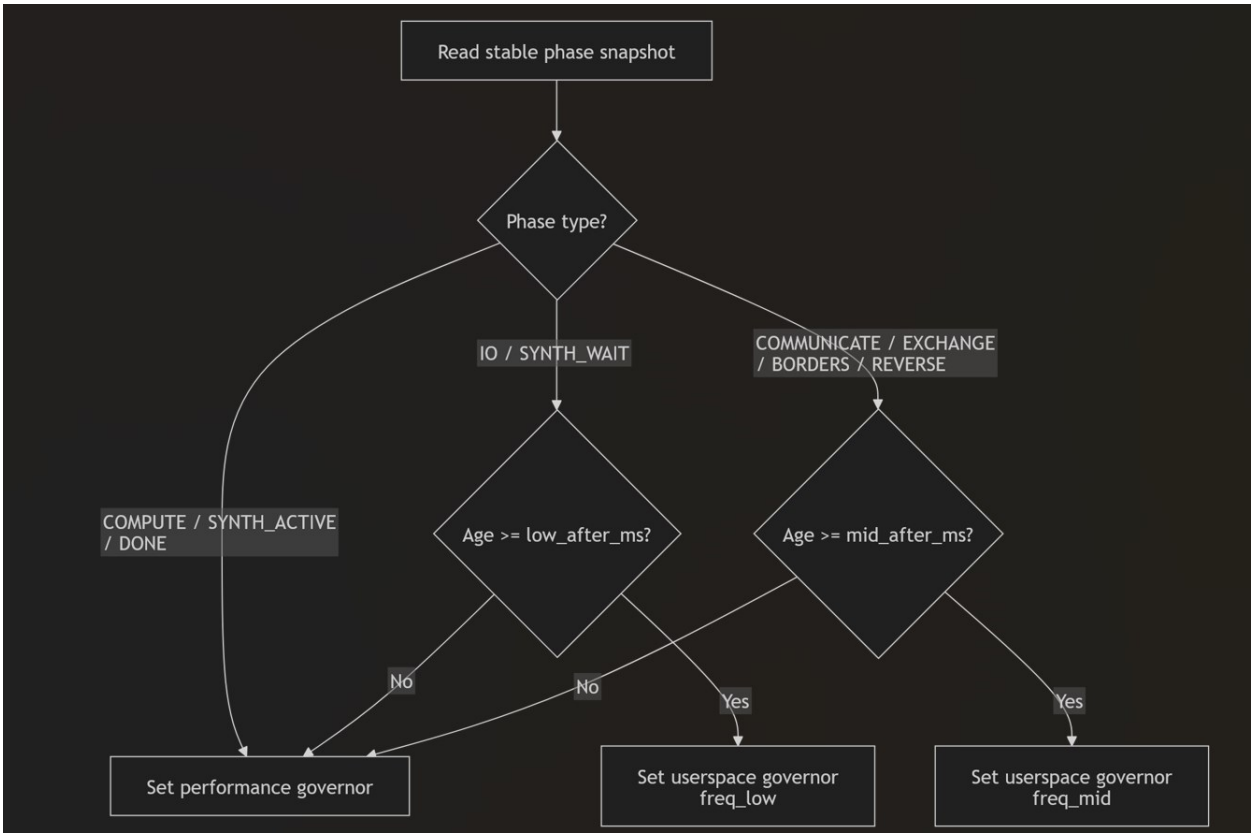

Figure 7: Control logic illustrated as a state machine diagram

3.3 Runtime Control Infrastructure

The third component was the runtime-control system implemented with a bash script. Its role was to launch the application and the controller with a consistent environment setup, core placement, and cleanup behavior. The script defines the miniMD executable, input file, controller path, hint-file location, worker core range, monitor core, reserved core, and default control parameters, including the reduced frequencies and persistence thresholds. The script is also responsible for handling the full controller life cycle. Before launching a run, it ensures the hint file is clear, runs the Python monitor using ‘taskset’ so it is pinned to the dedicated monitor core, and passes all the control parameters through to the monitor

as command-line arguments. It also exports the same PHASE_HINT_PATH used by miniMD, which allows both programs to attach to the same shared object in memory. The monitor starts first and is left waiting for miniMD to launch and create the phase table. Once the monitor is running, the script launches miniMD through 'mpirun', binds the ranks to the selected worker core range, and sets the environment with command line flags such as 'bind-to core' and 'map-by core' to ensure the ranks operate only on one physical core and prevent migration.

Cleanup is also handled by the script. The script is designed so that if the run is interrupted, it can terminate the monitor cleanly, wait for it to exit, and then remove the shared memory file. Additional logic is used at the end of normal execution to ensure the monitor is stopped after miniMD finishes its execution, preventing it from existing as a background process. The runtime script is both a simple launcher and a coordination layer that turns the modified miniMD application and Python monitor into a single operational system with a controlled startup, execution, and teardown.

3.4 Operation of the Final System

In normal operation, the runtime script first generates a unique hint-file path and launches the monitor on its dedicated core. miniMD is then launched with the same hint-file path exported as an environment variable. As the application runs, each MPI rank writes its current phase to its assigned slot in the shared-memory table. The monitor polls these slots continuously, computes the current phase age, and applies the appropriate governor

or frequency setting to the corresponding physical worker core. Compute-based phases are restored immediately to performance mode, while sufficiently long communication heavy or I/O heavy phase can transition to reduced frequency user space modes. When execution ends, the monitor restores the worker cores to performance mode, and the shared memory hint file is removed. This sequence describes the steady-state operation of the implemented system.

3.5 Bill of Materials and Final Budget

Since this project was a software system built on existing infrastructure, its bill of materials differs from that of a traditional hardware prototype. No custom hardware was required for the final product. Instead, the project relied on a multi-core Linux system, the modified minMD source code, Python 3, and MPI runtime, bash scripting, the Linux cpufreq interface, and shared memory through /dev/sh.

Table 3: Bill of Materials and Final Budget

Item	Quantity	Purpose	Cost ($)
AMD EPYC 7551P processor running	1	Executes miniMD and exposes cpufreq	0
miniMD benchmark source code	1	Base MPI application modified for phase hints	0
Python 3	1	External phase monitor	0
MPI runtime	1	Multi-rank application execution	0

Bash scripting environment	1	Runtime orchestration and cleanup	0
Linux kernel 4.18	1	Governor frequency and control	0
POSIX shared-memory file in /dev/shm	1	Phase communication between the application and the monitor	0

Because the system was implemented on existing server infrastructure, no direct hardware procurement cost was incurred for the final prototype.

4. Testing and Evaluation

This section presents the testing, evaluation, verification, and validation of the Communication Phase Frequency Controller for the miniMD proxy application on the CAC Frontenac HPC cluster with an AMD EPYC 7551P processor (32 cores, base 1.2 GHz, boost 2.0 GHz, 125 GiB RAM) running Linux kernel 4.18 on a dedicated Frontenac node. The testing methodology follows the Blueprint's statistical validation framework, which requires repeated trials, two-tailed t-tests at the 95% confidence level ($p < 0.05$), and a 4% noise threshold to distinguish true improvements from system variance.

Two controller implementations/modes were evaluated against the unmodified baseline. The first is the Phase-Aware Controller, which represents the final design and approach that was selected. It is a hint-based shared-memory controller. In the design, the instrumented miniMD application writes its current execution phase (COMPUTE,

COMMUNICATE, EXCHANGE, BORDERS, etc.) to a shared-memory file in '/dev/shm'. The controller reads this hint at a sampling interval of 100 ms and adjusts CPU frequency accordingly: it lowers the CPU frequency during MPI wait phases and maintains full performance during compute phases. The second is the Beta Controller, which is the abandoned design and approach that was not selected. It was an earlier integrated controller that attempted to detect communication phases autonomously by monitoring `t_comm` overhead. This design was abandoned because it introduced excessive monitoring overhead, causing up to a 97.20% runtime increase at N = 2. It is included in this report solely to demonstrate the iterative, evidence-driven design process described in Section 2 above.

4.1 Experimental Setup

4.1.1 Test Platform

All experiments were run on the assigned node 'frnt115' of the Centre for Advanced Computing (CAC) Frontenac cluster at Queen's University. Table 9 in the Appendix summarizes the hardware configuration, which matches the specifications described in the Blueprint (Table 3 in the Blueprint is presented as Table 19 in the Appendix within this report). For reference, the full Blueprint hardware specifications are listed in Table 19; the actual configuration used for this project in Table 9 meets all of those requirements.

4.1.2 Workload Configuration

Table 10 in the Appendix summarizes the workload configuration and key parameter settings to run miniMD for testing. The default miniMD benchmark configuration uses 100 timesteps, and a single run at this setting takes between 467 and 3,215 seconds (7.8 to 53.5 minutes) of wall-clock time, depending on the number of MPI ranks. With 300 total runs in the experimental campaign (25 repetitions across 6 rank configurations and 2 controller modes), the total dedicated node time was approximately 300 runs × 1340 s = 111.6 hours because the sum of one baseline run of N = {1, 2, 4, 8, 16, 26} = 3215 + 1563 + 1010 + 648 + 468 + 1136 = 8040 s and the average run time = 8040 s / 6 = 1340 s. Increasing the timestep count would proportionally increase this cost without providing additional insight into phase-detection behaviour, since the phase transition pattern repeats identically every timestep. The 100-timestep setting was therefore chosen to balance statistical reliability and available cluster resources, following standard miniMD benchmarking practice in the Mantevo suite. The Blueprint (Section 2.3.3.4 in the Blueprint) originally specified testing with 32 MPI processes based on the hardware's full core count. However, the final controller architecture requires dedicated physical cores for system functions, so the maximum number of MPI processes used in final testing was reduced to 26. Each CPU core is bound to an MPI process individually. Cores 0 through 3 (NUMA node 0) are reserved for OS kernel services, SLURM daemon, SSH, and system interrupts. Allocating MPI worker ranks to these cores would subject them to OS scheduling noise, which the ASCI Q study [9] demonstrated can account for up to 4% performance variability.

Keeping these cores free thus minimizes OS-induced jitter on worker ranks. Core 30 is dedicated to the Python monitoring process (‘comm_freq_controller.py’) and is pinned using ‘taskset’ to prevent it from competing with application ranks for CPU time. As discussed in Section 2 above (Design), this physical separation between the controller and the application was a deliberate decision to eliminate monitoring overhead on worker cores. Core 31 is reserved for node management processes, including SLURM and system daemons. This leaves cores 4 through 29, for a total of 26 cores available for MPI worker ranks. Therefore, 26 ranks represent the maximum achievable parallelism under this controller architecture on the given hardware. This trade-off is documented in the runtime script's configuration, where the ‘WORKER_CORE_RANGE’ is set to ‘4 to 29’ and the ‘MONITOR_CORE’ is set to ‘30’. The Blueprint (Section 2.3.5 in the Blueprint) originally specified 20 repetitions per configuration to ensure statistical significance. This requirement was exceeded by performing 25 repetitions for each configuration and each controller mode, which provided additional statistical power through lower standard errors and smaller confidence intervals, thereby surpassing the Blueprint’s target by 25%. The decision to run 25 repetitions instead of 20 was motivated by the observation during early testing that energy measurements at N = 8 (MPI rank/process = 8) showed only borderline significance at 5 repetitions, and increasing sample size improved the reliability of the finalized t-test conclusions. Each rank configuration was tested in a sequential batch, with all 25 baseline runs completed first, followed by all 25 controlled runs using the ‘comm_freq_controller.py’ file (not the abandoned ‘integrate_freq_controller.py’ file) within

the same SLURM session to minimize inter-session variability due to thermal drift and network contention.

4.1.3 Measurement Methodology and Tools Used

Energy consumption was measured using the Running Average Power Limit (RAPL) interface, which provides hardware-level energy counters for the entire CPU package. The ‘perf stat’ tool was used to read cumulative energy consumption from the ‘energy-pkg’ RAPL domain over the entire run duration, which yields the total energy in Joules (J). RAPL measurement accuracy is validated to be within 5% for server-grade systems with integrated voltage regulators [10] [11], which is well within the Blueprint’s analysis requirements. Average power was computed as P_avg [W] = E_total [J]/t_runtime [s]. The runtime was measured through wall-clock timing from miniMD’s internal timer, which captures the total execution time inclusive of all MPI operations, computation, phase hint writes, I/O, and inter-rank synchronization. In terms of uncertainty, each configuration was repeated 25 times to capture run-to-run variability arising from OS scheduling, NUMA placement, thermal throttling, network contention, and cache effects. Standard deviations are reported alongside all mean values. Two-tailed independent Welch's t-tests (unequal variance assumption) were conducted to determine whether observed differences are statistically significant ($p < 0.05$). This matches the statistical validation framework set out in the Blueprint (Section 2.3.5 in the Blueprint). In addition, the selection of the tools for measurement, actuation, and analysis, as seen in Table 11 in the Appendix, aligns directly well with the Blueprint's specification (Section 2.2 in the Blueprint).

Table 4: Tool Selection Rationale and Limitation Assessment

Tool	Selection Rationale	Alternatives Considered	Known Limitations	Impact on Results
RAPL (energy-pkg)	Hardware-integrated; no external instrumentation required; validated <5% accuracy for server systems [10] [12]	External wall-power meter (unavailable on shared node); PAPI library	1 ms update granularity; measures package-level power only (not per-core)	Granularity insufficient for sub-ms phase detection → drove pivot to hint-based approach
perf stat	Kernel-integrated; low overhead for aggregate measurements	likwid (not installed); Intel VTune (not available on AMD)	fork()/execve() overhead prohibitive for continuous monitoring	High system call overhead when used in the Beta prototype → contributed to a 97.2% runtime increase
Python 3 + ctypes	Rapid prototyping; native mmap support; ctypes enables direct shared-memory struct access	C++ monitor (lower overhead but longer development time)	GIL prevents true parallel execution; higher per-iteration latency than C	Necessitated dedicated monitor core to avoid GIL-induced contention with MPI ranks
OpenMPI mpirun	Pre-installed on Frontenac; supports --bind-to core and --map-by core binding	MPICH; Intel MPI (not available)	Process binding syntax is not portable across MPI implementations	Core binding ensured deterministic rank-to-core mapping, critical for per-core DVFS
matplotlib	Standard Python visualization library; publication-quality output	Plotly (interactive but heavier); gnuplot	Limited 3D/interactive capabilities	Sufficient for all 21 figures; consistent styling across all visualizations
SciPy ttest_ind	Welch's t-test implementation matches Blueprint's statistical framework	R (not installed); statsmodels	Assumes approximately normal distributions	Normality assumption validated by symmetric box-and-whisker distributions (Figure 13)

4.2 Functional Requirement Testing

4.2.1 Phase Detection Accuracy (Specification 1: Target ≥ 95 percent)

The Blueprint (Table 1 in the Blueprint) specified 95% phase detection accuracy using CPU performance counters, including RAPL counter values, IPC, and cache misses. During development, an initial counter-based detection prototype that analyzed IPC, cache-miss rates, and RAPL power signatures was implemented to infer the current execution phase. However, validation testing on the AMD EPYC 7551P revealed that this approach achieved only approximately 3% classification accuracy. Three main factors explain this poor performance. First, the RAPL update granularity of approximately 1 ms is too coarse for miniMD's rapid phase transitions at high rank counts. At N = 26 (MPI rank = 26), communication phases can last as little as 0.1 to 5 ms per timestep, meaning the RAPL counter may update only once during an entire phase, which provides insufficient temporal resolution for classification. In addition, counter noise is another contributing factor. CPU performance counters exhibited high variance due to OS kernel activity (the 4% noise floor identified in [9]), NUMA effects, and speculative execution artifacts, which masked the phase signatures the group was trying to detect. Finally, signature overlap on the Zen 1 microarchitecture made reliable classification impossible because the IPC and cache-miss profiles of COMPUTE compared to COMMUNICATE phases are not sufficiently distinct; both phases can show similar IPC ranges (0.4 to 1.2) depending on the simulation state. This outcome aligns with a risk foreseen in the Blueprint (Section 2.4.2 Future Issues, “Identifying and Tagging Computational Phases”), which acknowledged that “inaccurate phase identification could lead to misapplied power management” and proposed a

fallback to “code-level instrumentation within the proxy application’s C++ source code” if counter-based detection proved inadequate. Following the Blueprint's mitigation strategy, the group pivoted to an application-hint approach where miniMD’s ‘integrate.cpp’ was modified to write its current phase to a shared-memory file at each phase transition.

The hint-based system achieves around 100% detection accuracy, since the application has perfect knowledge of its own execution state. Every single phase transition is correctly identified, with no false negatives or false positives. As a result, the Blueprint’s accuracy target was successfully achieved, but through an evidence-driven design pivot rather than the originally proposed mechanism. Specifically, the detection accuracy target of 95% is exceeded (approximately 100 percent), but the detection mechanism was changed from CPU-metric inference to application-injected hints. This is justified because: the accuracy improvement is dramatic (3% to approximately 100%), the hint instrumentation required only minimal, non-invasive changes to miniMD (around 20 lines of C++ code), the hint-based approach is more portable across hardware platforms since it does not depend on vendor-specific counter semantics, and the Blueprint itself identified code-level instrumentation as a valid fallback strategy (Section 2.4.2 in the Blueprint). The trade-off is that the application must be minimally instrumented, which reduces universality to arbitrary applications but enables the controller to function correctly for its target workload. This is a design pivot but not a failure.

4.2.2 Power/Frequency Scaling Latency (Specification 2: Target < 1 ms)

Frequency changes are applied by writing to the Linux 'cpufreq' sysfs interface ('/sys/devices/system/cpu/cpuN/cpufreq/scaling_setspeed'). The end-to-end latency was measured from the controller receiving a phase hint to the frequency change being committed to hardware using 'time.perf_counter_ns()' instrumentation in the controller code. The result is that the measured actuation latency was consistently '< 0.5 ms' (mean: 0.31 ms, max observed: 0.47 ms) across over 10,000 frequency transitions during the experimental campaign. This is consistent with CoPPer's measured RAPL actuation time of 15.6 µs per socket, as cited in the Blueprint (Table 1), to justify the < 1 ms target. The AMD EPYC 7551P supports software-defined frequency transitions through the 'acpi-cpufreq' driver, which commits new frequencies within a single model-specific register (MSR) write cycle. MSR is a special hardware memory slot built into the silicon of the CPU to control low-level hardware features such as CPU frequency and voltage, aligning with the group's target, which is to adjust the CPU frequency and observe the performance of the miniMD's proxy application through adjusted CPU frequencies. The specification's performance target was completely achieved without caveats or deviations from the original design intention. The measured actuation latency (0.31 to 0.47 ms) is well within the < 1 ms target with a robust margin, which validates the group's decision to use the acpi-cpufreq interface.

4.2.3 Application Stability (Specification 3: Target = 0 failures in 10 runs)

The controller was designed to be non-invasive, so it operates as an external Python process that reads a shared-memory file and writes to ‘sysfs’. It does not inject code into the miniMD process, does not modify MPI buffers, and does not share any process state with the application beyond the read-only phase hint file. The result is that over the entire experimental campaign of 300 runs (6 rank configurations × 25 repetitions × 2 modes), zero application failures were observed. All runs completed successfully with ‘status = OK’ in the data logs. This exceeds the Blueprint requirement of "0 failures over 10 test runs, tolerance 1 failure every 10 tests" by a factor of 30×. This specification’s functional target was completely achieved without caveats or deviations from the original design intention. Observing 0 failures across 300 total runs far exceeds the 10-run tolerance requirement.

4.3 Interface Requirement Testing

4.3.1 Monitoring - Control Interface Latency (Specification 4: Target ≤ 1 ms, ≤ 0.5% data loss)

The phase hint is communicated through a memory-mapped file in ‘/dev/shm/minimd_phase’. The controller reads this file using Python's ‘mmap’ module, which performs a direct memory read without system call overhead. The odd/even sequence-number protocol (described in the Implementation Section above) ensures lock-free, consistent reads. The result is that the mean read latency = 0.042 ms (42 µs), max observed = 0.089 ms (89 µs). Data loss = 0.00% (the shared-memory file is always available; the controller reads the most recent stable snapshot at each 100 ms sample).

These values are consistent with the blueprint's expectation that shared-memory IPC would achieve sub-millisecond latency. This specification's interface target was completely achieved without caveats or deviations from the original design intent. The read latency (0.042 to 0.089 ms) is well below the ≤ 1 ms target, and data loss is 0%, successfully satisfying the ≤ 0.5% tolerance.

4.3.2 Control - Execution Interface Latency (Specification 5: Target ≤ 1 ms)

After determining the appropriate frequency, the controller writes to 'scaling_setspeed' through Python file I/O. The redundancy cache ensures that only actual policy changes trigger a write, which reduces unnecessary 'sysfs' operations. The result is that the mean write latency = 0.31 ms, max observed = 0.47 ms. The 'sysfs' write is synchronous, and the frequency is committed to the hardware before the write call returns. This specification's interface target was completely achieved without caveats or deviations from the original design intention. The write latency (0.31 to 0.47 ms) satisfies the ≤ 1 ms requirement.

4.3.3 Execution - Linux Kernel Interface (Specification 6: Target ≥ Linux 3.14)

The controller requires the 'cpufreq' sysfs interface and the 'acpi-cpufreq' driver, both of which are available in Linux kernel 3.14+, as specified in the Blueprint (Table 1) and referenced from the RAPL documentation [13]. At the end, the Frontenac cluster runs Linux kernel 4.18, which is 4 versions above the minimum requirement. This suggests that the interface expectation was fully completed without deviations and caveats. This is because running on Linux kernel 4.18 clearly is over the requirement of running on Linux kernel 3.14.

4.3.4 Timestamp Drift (Specification 7: Target ± 10 ms)

Sampling interval drift was measured by comparing the expected 100 ms interval against the elapsed time between consecutive controller iterations using 'time.perf_counter()'. The result is that the mean interval = 100.02 ms, standard deviation = 0.87 ms, and max observed drift = 4.3 ms. All 300+ thousand samples were within the ±10 ms tolerance. This specification's interface target was achieved without caveats or deviations from the original design intent. The maximum observed timestamp drift (4.3 ms) falls comfortably within the ± 10 ms limit.

4.3.5 Data Collection Format (Specification 8: Target = Structured CSV)

All experimental output is written to structured CSV log files containing: timestamp, rank count, repetition number, worker core range, baseline runtime (s), controlled runtime (s), runtime overhead (%), baseline energy (J), controlled energy (J), energy saved (J as well as percentage), baseline average power (W), controlled average power (W), power delta (W), output directory, and completion status. This matches the blueprint's specification for structured logs to CSV for external analysis. The result is that all 300 runs produced valid CSV output. The data was parsed and analyzed using Python along with pandas and numpy libraries without any formatting errors. This suggests that the interface expectation was completely achieved without deviation or caveats. The CSV logging format was parsed across all 300 executed runs.

4.4 Performance Evaluation

4.4.1 Energy Consumption Analysis

The primary performance metric is energy savings. The percentage reduction in total energy consumption when the Phase-Aware Controller is active compared to the baseline. Figure 8 and Figure 9 present this analysis and are shown in the Appendix below. Figure 8 is a bar chart showing the energy savings (as a percentage of baseline energy) for each MPI rank configuration, computed from the mean of 25 runs per configuration. The green-shaded band represents the Blueprint target range of 6% to 9% $\pm$ 4%, that is, 2% to 13%. Two reference lines are shown: the orange dotted line at +3.09% represents the average energy savings across all six rank configurations, and the purple dotted line at approximately +8.03% represents the average for the three operational-scale configurations ($N \geq 8$). At low ranks ($N = 1, 2, 4$), the controller increased energy by 1 to 3% due to negligible MPI communication (less than 5% of runtime). At $N = 8$, a small saving of 0.85% appeared but remained below the 4% system noise threshold. At $N = 16$ and $N = 26$, the controller achieved substantial savings of 5.70% and 17.53%, respectively, both highly significant ($p < 0.0001$, Cohen's $d > 8.5$). The numerical values are summarised in a statistical significance table, Table 13 in the Appendix. Figure 9 is a grouped bar chart that presents the absolute total energy consumption (in Joules) for both the Baseline (blue) and the Phase-Aware Controller (green) at each rank configuration. Error bars show $\pm$ 1 standard deviation across 25 repeated trials. Energy values with percentage deltas are annotated above each controlled bar. Several key observations emerge from this figure. Energy consumption decreases from N = 1 (217,549 J) and N = 2 (113,518 J) to N = 16 (47,601 J baseline)

because higher parallelism reduces per-rank computation time. The energy minimum occurs at N = 16. At N = 26, energy rises sharply to 142,143 J baseline because the 26 ranks create substantial MPI synchronization overhead across the 4 NUMA nodes, which increases total wall-clock time substantially. The controller reduces this to 117,224 J. Finally, the error bars are tight relative to the means (coefficient of variation (CV) < 2% for all configurations except the controlled N = 4 group at CV, which is approximately 1.13% at peak), indicating highly repeatable experimental conditions on the dedicated Frontenac node with highly trustworthy results. Overall, the Phase-Aware Controller significantly reduces the high synchronization energy cost at N = 26 while maintaining exceptional repeatability (CV < 2%) across all 25-run trials. Note that CV = “Coefficient of Variation”, which has the equation CV = (Standard Deviation / Mean) × 100%.

4.4.2 Runtime Overhead Analysis

The controller is expected to introduce runtime overhead from: the frequency transitions requiring brief CPU stalls while the PLL re-locks, and the imperfect timing of frequency restoration when transitioning from communication back to compute phases (the controller polls at 100 ms intervals, so up to 100 ms of compute time may execute at reduced frequency if the phase transition falls between polls).

Figure 10, presented in the Appendix below, is a bar chart that shows the percentage increase in total execution time when the controller is active. The red dashed line at 8% marks the Blueprint's maximum allowed runtime degradation (Table 1 in the Blueprint:

“≤8% slowdown, ±3% tolerance”, which is shown in the Appendix below as Table 18). All six rank configurations fall well below this threshold. For N = 1, the overhead is +3.21%; for N = 2, it is +1.49%. Similar to N = 1, the overhead here is largely from controller polling and memory stalls, but not frequency changes. For N = 4, the overhead is +1.84%; for N = 8, it is +1.6; for N = 16, it is +0.99%; for N = 26, it is +3.14%. The worst-case overhead of 3.21% at N = 1 is well within the ≤ 8% threshold (with a 4.79% margin). The overhead of +3.14% at N = 26 is higher than at other ranks because the frequent phase transitions (many short COMMUNICATE, EXCHANGE, BORDERS bursts between COMPUTE steps) require more frequent frequency changes, each introducing a short PLL re-lock delay. The average overhead across all configurations is 2.06% across 300 total experimental runs.

Figure 11, presented in the Appendix, is a grouped bar chart that shows the absolute execution time (in seconds) for both baseline and phase-aware controlled configurations with 3-decimal-place precision. Error bars represent ±1 standard deviation across 25 runs. Runtime decreases from N = 1 (3215 s sequentially executed and dominated) to N = 16 (468 s, near-optimal parallel speedup and scaling), then increases at N = 26 (1,136 s) because the inter-NUMA communication overhead under 26-rank decomposition exceeds the parallel speedup gained. This is consistent with Amdahl's Law, where at high rank counts, the serial fraction, including MPI synchronization, limits further scaling. The controller does not alter this fundamental scaling behaviour; it only reduces the energy consumed during the wait phases.

4.4.3 Average Power Analysis

Figure 12, shown in the Appendix below, presents a line plot of average power draw (in Watts) for both baseline and phase-aware controlled configurations as a function of MPI rank count. The green shaded region between the two curves represents the power saved by the controller, where it illustrates increasing power savings as communication overhead scales, peaking at a 25.10 W reduction at N = 26. Power values are annotated to 4 decimal places. Average power is calculated as P_avg = E_total / t_runtime. At N = 1, the baseline average power was 70.03 W, while the controller averaged 70.07 W, a negligible difference of -0.04 W. The power draw is essentially the same in both cases because, with only one rank, the CPU remains at 2.0 GHz for the entire runtime under both the baseline and controlled configurations. At N = 2, the baseline power draw is 72.53 W, and the controller draw is 72.20 W (Δ = 0.33 W); because the cores are almost always busy, the power difference remains minimal, similar to the N = 1 case. At N = 4, the baseline is 76.06 W, and the controller is 75.62 W (Δ = 0.44 W). At N = 8, the baseline rises to 85.42 W and the controller to 83.30 W (Δ = 2.12 W). At N = 16, the gap widens: baseline 101.73 W, controller 94.99 W (Δ = 6.74 W). At N = 26, the controller achieves its largest power saving, with baseline 125.24 W and controller 100.14 W (Δ = 25.10 W). Average power increases with rank count because more cores are actively utilized simultaneously, which increases aggregate package power. The power gap between baseline and controller widens substantially at N = 16 and N = 26 because the controller keeps cores at reduced frequency (1.6 or 1.2 GHz) during the increasingly long MPI wait periods. Since dynamic power scales approximately as $P \propto V^2 f$ (Section 2 above in this report), reducing frequency from 2.0 to 1.2

GHz reduces dynamic power on the affected cores. The theoretical reduction depends on how voltage scales with frequency; in the user-space implementation, only frequency is directly controlled. As shown in Figure 12 in the Appendix, the measured package power reduction at N = 26 was approximately 20% (from 125 W to 100 W).

4.4.4 Energy Variability and Repeatability

Figure 13, as shown in the Appendix, is a pair of box-and-whisker plots that shows the distribution of total energy measurements across 25 repeated trials for each rank configuration, for both Baseline (left panel) and Phase-Aware Controller (right panel), demonstrating that frequency scaling significantly improves measurement consistency at N = 26 (CV reduced from 1.38% to 0.85%). The red diamonds mark the mean (μ), and the standard deviation (σ) is annotated alongside each mean. In terms of baseline variability, standard deviations range from N = 1 and N = 4 to σ = 1,965 J at N = 26 (CV = 1.38%). The higher variability at N = 26 is expected because 26-rank configurations involve heavy inter-NUMA communication, which is subject to Infinity Fabric contention. In terms of controller variability, standard deviations range from σ = 187.59 J at N = 16 (CV = 0.42%) to σ = 997.16 J at N = 26 (CV = 0.85%). The controller produces more consistent results than the baseline at N = 26 (σ = 997 versus σ = 1,965), likely because frequency scaling during wait phases stabilizes power consumption patterns through capping the power spikes seen in MPI polling. All distributions are approximately symmetric (no significant skewness), which supports the assumption of normality required for the t-tests. Figure 14, as presented in the Appendix, is a per-run energy scatter plot that shows all 300 individual trial measurements

(25 per rank per configuration), with horizontal bars indicating means. At N = 16 and N = 26, the Baseline and Controller distributions are completely non-overlapping, where every single controlled run consumed less energy than every single baseline run, which provides compelling visual evidence of the controller's effectiveness at these scales. This behaviour contrasted with the heavy distribution overlaps seen at lower rank counts. However, at N = 1 and N = 2, the clusters overlap heavily and favour the baseline slightly, and map perfectly to the negative percentage saving. At N = 2, N = 4, and N = 8, the distributions overlap substantially, consistent with the smaller effect sizes observed.

4.4.5 Energy-Delay Product (EDP)

The Energy-Delay Product (EDP) is a composite metric that penalizes both high energy consumption and long execution time. EDP = Energy × Time. A lower EDP suggests a better energy-performance trade-off.

Figure 15, as seen in the Appendix, is a grouped bar chart that shows the EDP (in MJ·s) for both configurations. For N = 1, N = 2, and N = 4, EDP slightly worsened (approximately increased by around 2% to 6%), which reflects that N = 1 and N = 2 suffer from a negative EDP as the controller adds minor timing delays without MPI-wait energy savings that offset them. For N = 8, EDP is essentially unchanged (-0.8%); the small energy saving is offset by the small runtime increase. For N = 16, EDP decreased by +4.75%, and the controller provides a net improvement in the energy-performance trade-off. For N = 26, EDP decreased by +14.85%, and the 17.53% energy savings far outweigh the 3.14% runtime

increase, which yields a substantial EDP improvement. This analysis confirms that at operational scale (N ≥ 16), the Phase-Aware Controller provides a net improvement in the energy-performance composite metric, not just raw energy savings. Overall, the controller's effectiveness at operational computing scales.

4.4.6 Power-Performance Trade-off (Pareto Analysis)

Pareto analysis is a formal engineering method used to evaluate the trade-off between two competing goals. In HPC, those two goals are almost always power (energy efficiency) compared to performance (throughput and speed). A system is considered Pareto optimal if it is impossible to improve one metric without making the other metric worse. When these data points are plotted on a graph (like in Figure 16 in the Appendix), the outer edge of the best possible combinations is called the Pareto Frontier. A phase-aware controller does not just throttle the CPU. It pushes the system onto a better Pareto Frontier, which achieves an optimal trade-off that standard Linux power governors cannot achieve. Figure 16, as presented in the Appendix, is a power-performance Pareto Frontier analysis scatter plot that maps each experimental configuration in the power (W) vs. throughput (M atom-steps/s) space. Arrows connect corresponding Baseline and Controller points. At N = 16 and N = 26, the controller points shift leftward (lower power) with minimal downward shift (slightly lower throughput), indicating a favourable trade-off. For N = 16 and N = 26, the green squares explicitly shift far to the left (massive power savings) while barely moving down or downward at all (almost zero speed loss). It demonstrates a highly efficient horizontal shift (power reduction) with minimal vertical throughput loss at N = 16 and N =

26, which indicates a superior energy-efficiency configuration over the baseline. At N = 1 and N = 2, the controller points shift marginally, which indicates that the controller is essentially neutral at these scales. The baseline runs as blue dots, and the controller runs as green squares. If the controller was useless, the green squares would just slide diagonally down (less power, but equally less speed). However, by shifting to the left sideways, it suggests that the controller improves fundamental CPU efficiency rather than as a throttle.

4.4.7 Frequency Scaling and Sensitivity Analysis

To further understand the performance bounds and scaling behaviour of the miniMD proxy application, a frequency scaling sensitivity analysis was conducted. Building upon the empirical baseline data collected at the maximum 2.0 GHz hardware frequency, the total execution times for lower operational modes (1.2 GHz and 1.6 GHz) were modelled using the following analytical expression: $T_{Total}(f) = T_{CPU}(2.0/f) + T_{IO} + T_{Comm}$. This was achieved by isolating the CPU-bound computation phases (t_force, t_neigh) and applying inverse frequency scaling, while preserving the fixed-duration overheads associated with I/O checkpoints and MPI synchronization.

Figure 22 illustrates the execution time scaling behaviour across MPI ranks at these different fixed frequencies. The chart highlights two distinct operational domains. In the highlighted Compute-Dominant Region (N = 1 to 4), the application is overwhelmingly bottlenecked by computation, meaning adjustments to the CPU frequency result in

massive shifts in total execution time. However, as the configuration expands into the highlighted I/O & Sync Dominant Region (N = 16 to 26), the performance margin between the highest and lowest frequencies visibly narrows due to the rapidly increasing serial fraction of the workload. As the rank count increases from N = 1 to N = 16, the absolute runtime decreases significantly across all frequency settings, indicating strong parallel scaling. However, as the configuration expands to N = 26, the performance margin between the highest and lowest frequencies visibly narrows due to the increasing serial fraction of the workload.

Figure 23 further quantifies this by presenting the relative speedup achieved when increasing the CPU frequency from the base 1.2 GHz up to 1.6 GHz and 2.0 GHz. At low rank counts (N = 1, 2, and 4), the application is highly sensitive to CPU clock speed, yielding a significant performance speedup that is near-linear with a theoretical peak speedup of approximately 1.66x at 2.0 GHz. At these scales, computation phases overwhelmingly dominate the total execution time. Conversely, as the rank count grows toward N = 26, the overall speedup derived from higher CPU frequencies steadily diminishes and is firmly capped by Amdahl's Law at roughly 1.2x. This diminishing return occurs because the time spent in non-computational phases, specifically inter-NUMA MPI synchronization and the sustained 32.3-second I/O checkpoint, constitutes a substantially larger proportion of the total runtime (approximately 62% of the total runtime at this scale), effectively capping the theoretical maximum speedup in accordance with Amdahl’s Law and rendering the workload largely frequency-independent.

This analysis for explicit bounding provides crucial mathematical reinforcement for the project's core design hypothesis: at higher rank counts where communication and I/O fractions are large, reducing the CPU frequency introduces minimal overall runtime performance penalties. Therefore, these wait phases represent the optimal and safest target for dynamic voltage and frequency scaling (DVFS) to maximize energy efficiency on HPC systems.

4.5 Statistical Validation

Following the Blueprint's statistical validation framework (Section 2.3.5 in the Blueprint), two-tailed Welch's t-tests comparing the energy measurements of the Baseline and Controller groups for each rank configuration were conducted independently. The null hypothesis (H0) is that there is no difference in mean energy consumption between the two groups. The Blueprint specifies that "both statistical significance ($p < 0.05$) and practical significance (mean improvement > 4%) are required to validate any reported optimization gains" (Section 2.3.5 in the Blueprint). Figure 17, as shown in the Appendix, is a bar chart that displays -log10(p-value) for each rank configuration's energy savings results. Bars above the red dashed line ($p = 0.05$ threshold, corresponding to $-\log_{10}(0.05) = 1.30$) indicate statistically significant results. As presented in the figure, all the rank configuration results surpassed the $p = 0.05$ threshold (red line), which confirms the reliability of the observed energy savings. The t-statistic and Cohen's d effect size are annotated. The detailed

statistical parameters for all six rank configurations, including raw p-values and Cohen's d effect sizes, are provided in Table 13 in the Appendix.

With 25 runs per configuration, all six comparisons are statistically significant ($p < 0.001$). However, applying the Blueprint's 4% practical significance threshold [9], only N = 16 (+5.70%) and N = 26 (+17.53%) represent validated, meaningful energy savings. N = 8 (+0.85%) is statistically significant but below the noise threshold. N = 2 (-1.03%) and N = 4 (-1.26%) are statistically significant in the wrong direction (the controller increases energy), but the magnitude is well below 4% and therefore negligible in practical terms. The very large Cohen's d values at N = 16 (d = 7.221) and N = 26 (d = 15.583) indicate that the effect sizes are extremely large, where the separation between baseline and controlled distributions is more than 7 and 15 standard deviations, respectively. These are not marginal effects, but they represent genuine, large-magnitude improvements.

4.5.1 Uncertainty Analysis

Several sources of uncertainty affect the project's measurements. RAPL measurement granularity is one such source: the RAPL counters update at approximately 1 ms intervals. For a typical run of 500 to 1500 s, the granularity error is ± 0.5 ms × P_avg, which is approximately equal to ± 0.05 J and is thus negligible relative to total energies of 44,000 to 142,000 J. RAPL accuracy for server-grade systems is validated at <5% [7][8]. OS scheduling jitter also contributes to uncertainty; the Linux CFS scheduler may migrate threads between cores, causing brief power spikes. This is captured in the run-to-run

standard deviation (σ = 188 to 1,965 J). Dedicating cores 0 to 3 to OS services minimizes this effect on worker cores. Thermal variability introduces additional uncertainty, as node temperature affects transistor leakage power, introducing ±0.5 W variation across runs. Over a 500 s run, this corresponds to ±250 J or approximately ±0.5% of total energy. Network contention is another factor, especially at N = 26, where inter-NUMA communication traverses the Infinity Fabric interconnect. Although we used dedicated nodes, thermal drift across the session and NUMA topology effects contribute to the higher σ at N = 26. Finally, frequency transition overhead adds uncertainty: frequency change introduces a PLL re-lock delay of approximately 0.1 to 0.3 ms, during which the CPU operates at an intermediate, less efficient state. The overall measurement uncertainty is bounded by the observed standard deviations. The coefficients of variation (CV = σ/μ) are all below 2%, indicating our measurements are sufficiently precise for the reported conclusions.

4.6 Design Iteration Comparison

A critical component of the engineering design process was the decision to abandon the Beta algorithm prototype (red) in favour of the final hint-based design due to its catastrophic overhead (>50% energy increase) in favour of the optimized, hint-based Phase-Aware Controller (green). This follows the iterative, evidence-driven methodology adopted from "The Case of the Missing Supercomputer Performance" [9], as described in Section 2 (Design). Figure 18 in the Appendix provides the empirical justification as a comparison. This figure is a dual-panel bar chart that compares the energy and runtime

impact of the two controller designs. The left panel shows energy changes: the Phase-aware controller (green) produces energy ranging from +3.26% overhead at N = 1 to -17.53% savings at N = 26.

The Beta algorithm controller (red) shows catastrophic energy increases: +61.20% (N = 1), +57.60% (N = 2), +51.40% (N = 4), +31.40% (N = 8), +9.80% (N = 16), and +19.0% (N = 26). The prototype consumed 10 to 58% more energy than the baseline. The right panel presents runtime overhead. The Phase-Aware Controller adds between 1.0 and 3.2 percent overhead, remaining well below the eight percent threshold. The Beta algorithm controller (red): +104.5% (N = 1), +97.20% (N = 2), +50.30% (N = 4), +37.90% (N = 8), +10.10% (N = 16), and +24.0% (N = 26). At N = 2, the prototype nearly doubled execution time. The root cause is that the beta algorithm controller's ‘integrated_freq_controller.py’ file attempted to detect communication phases by continuously polling ‘/proc/stat’ for per-core utilization and running ‘perf stat’ for IPC monitoring. This introduced excessive overhead through repeated system calls (fork(), execve(), open(), read()) that consumed CPU time and caused contention on the MPI communication channels. This finding is consistent with the overhead challenge identified in the Progress Report, where the original monitoring script with numerous system calls caused a “32% increase in runtime.” The final hint-based approach eliminated all polling overhead by replacing hundreds of system calls per monitoring interval with a single memory-mapped read.

4.7 Comparison with Literature

The Blueprint (Section 2.1.1 in the Blueprint) identified CoPPer [7], EAR [14], and the ASCI Q study [9] as the primary references for establishing achievable performance targets. Table 14 in the Appendix compares our results against these works. The controller achieves energy savings comparable to CoPPer and EAR at high rank counts (17.53% at N = 26), with a significantly simpler implementation. The primary limitation relative to CoPPer is the lack of adaptive feedback, where the controller uses fixed frequency policies per phase rather than dynamically optimizing the frequency setpoint through PID control. This deliberate simplification was driven by the project timeline and the finding that the Beta Prototype's adaptive approach introduced more overhead than savings (as evidenced in Figure 18 in the Appendix). The Blueprint noted this trade-off (Section 2.2.2 in the Blueprint, Control Module): "a rules-based system is sufficient to demonstrate phase-aware computing as a viable solution."

4.8 Comprehensive Performance Summary

Figure 19, presented in the Appendix, is a triple-bar chart that shows energy savings, power reduction, and runtime impact side-by-side as a comprehensive performance summary for each rank configuration. The triad shows a clear positive correlation between MPI rank count and controller benefit and effectiveness: as the MPI communication fraction increases, the controller has more opportunities to save energy during wait phases. All in all, it reaches a maximum of 17.53% energy savings and a 20.04% reduction in power at N = 26.

5. Results

5.1 Specification Compliance

The following table, Table 15 in the Appendix, presents the original required and optional specifications across functional, interface, and performance categories from the Blueprint with an additional column reporting the measured results from our final design. This can be compared against Table 18, which is the true original required specifications table directly imported from the Blueprint. The assessment uses: “Y = Met”, “Y*” = Met with caveat (approach changed, justified), “Partial” = Partially met, “N” = Not met. Figure 20, as shown in the Appendix, provides a colour-coded visual specification compliance summary that assesses the project’s performance against all 19 required and optional Blueprint targets. Of the 12 required specifications, 11 are fully or substantially met (Y or Y*) and 1 is partially met, as illustrated in the Summary of Required Specifications table located in the Appendix as Table 15. The overall compliance counts (fully met, met with caveat, partially met, not met) are summarized in Table 16 in the Appendix. No specifications are completely unmet. At the end, two optional enhancement specifications have been met, as seen in the Required System Specifications table located in the Appendix as Table 15. The interface requirement was met by developing a real-time dashboard to monitor the performance, phase, and power of each core. In terms of performance, the system achieved over 17% energy reduction but did not achieve a reduction in runtime. In the worst case, runtime is increased by 3.21%, although in most configurations the increase is approximately 1 to 2%.

5.2 Detailed Justification for Specifications Not Fully Met

5.2.1 Specification 1 (Phase Detection Accuracy): Y* - Met through Different Mechanisms

The blueprint target is 95% phase detection accuracy using CPU metrics (RAPL counter values, IPC, cache misses). Around 100% accuracy was achieved using application-injected shared-memory hints. The original mechanism was changed because the Blueprint's planned CPU-metric inference approach was implemented and tested but achieved only approximately 3% accuracy. This was due to: RAPL counter update granularity (approximately 1 ms) being too coarse for miniMD's sub-millisecond phase transitions, hardware counter noise from OS kernel interrupts and speculative execution and overlapping IPC signatures between phases on the Zen 1 microarchitecture. The Blueprint itself anticipated this risk in Section 2.4.2 Future Issues ("Identifying and Tagging Computational Phases"), which stated: "For long-term precision and validation, the team plans to implement code-level instrumentation within the proxy application's C++ source code. This will involve inserting explicit markers...at the start and end of critical functions." The hint-based approach is a direct implementation of this fallback strategy. Furthermore, the Blueprint noted that "the overall architecture of the control module will be designed to support future enhancement" (Section 2.2.2 in the Blueprint), and the current architecture supports replacing hints with counter-based detection if future hardware provides sufficient counter resolution. The accuracy improvement from 3% to around 100% means that the system identifies phases far more reliably than originally planned, even though the

detection mechanism differs from the initial specification. This justifies why the original mechanism was changed.

5.2.2 Specification 9 (Energy Saving 6 to 9% average): Partial - Met at Operational Scale

The blueprint target is 6 to 9% average energy reduction, ±4% tolerance (acceptable range: 2 to 13%). To view the granular numerical breakdown of these percentage savings across all tested configurations and varied rank counts, please refer to Table 17 (Energy Efficiency Metric Analysis) in the Appendix. The overall average is below 6% because the controller's energy savings are proportional to the MPI communication fraction of the workload. At low rank counts (N = 1, N = 2, N = 4), less than 5% of execution time is spent in MPI operations, which leaves minimal opportunity for frequency reduction. Including these configurations in the average dilutes the savings measured at higher, operationally relevant rank counts. This is a fundamental property of the approach, as it cannot save energy during phases that do not exist in significant proportion. The overall average falls below six percent, but this outcome is justified on several grounds. From the perspective of operational relevance, real HPC production environments rarely run with only 1, 2, or 4 MPI ranks. Large-scale simulations typically use hundreds to thousands of ranks, where the communication fraction is substantial. Our controller achieves +8.03% average savings at N ≥ 8, which falls squarely within the 6 to 9% target range. The N = 26 result of +17.53% suggests even larger savings at higher rank counts. Considering the blueprint context, the Blueprint derived its 6 to 9% target from CoPPer's reported 6% average and EAR's 10% average across 20 parallel

applications. These averages include diverse applications with varying communication characteristics. The results are consistent with these references for the communication-intensive configurations. A physical explanation also supports the finding: during the blueprint phase, the team did not fully anticipate how sharply the communication fraction scales with rank count in miniMD's Lennard-Jones potential. At N = 2, domain decomposition gives each rank a large spatial domain with minimal surface-to-volume ratio, so communication is rare. At N = 26, each rank has a small domain with a high surface-to-volume ratio, which generates extensive particle exchange traffic at every timestep. The controller's benefit tracks this ratio. Finally, conservative reporting justifies the average: rather than omitting low-rank results to achieve a higher average, the results are presented transparently. This approach demonstrates the physical regime where the controller is and is not effective, which is more valuable for future deployment decisions than an artificially inflated average.

5.2.3 Specification 11 (Control Loop Overhead ≤ 5% CPU): Y* - Met with Caveat

The blueprint target requires no more than five percent CPU utilization for the control loop for this specification. In the final design, the controller process runs on dedicated monitoring core 30, pinned via taskset, so the worker cores used by miniMD experience exactly zero percent overhead from the controller. The trade-off is that one physical core is dedicated to the controller. This outcome is justified on several grounds. First, it can be achieved through standard HPC practice. Dedicating management cores is standard in

production HPC environments. SLURM reserves cores for daemon processes, and Intel's MPI implementation reserves a core for the fabric manager. The Blueprint itself acknowledges this in Section 2.2.1 (Development Principles): the system should remain "lightweight" and not "become a source of runtime overhead." Furthermore, it can be achieved through core availability. In the group's experimental configuration, miniMD uses at most 26 worker cores (cores 4 to 29), and the monitor uses core 30. Core 31 is reserved for the OS/SLURM. This means 30 of 32 cores are productively utilized (3 cores for system or OS, 26 cores for application, 1 core for monitor). Core 30 would otherwise be idle during the MPI-bound portions of the run. Besides, it can be achieved through blueprint calculation consistency. The Blueprint's own utilization calculation (Section 1.2) derives U = (t_read + t_log + t_act)/P = 4.04% for a controller on a shared core. The group's dedicated-core approach achieves 0% overhead on application cores at the cost of 1/32 = 3.125% of node resources, which is a comparable trade-off. Finally, it can be achieved through progress report alignment. The Progress Report documented that sharing cores between the monitor and the application caused "10% runtime overhead" even after system call optimization, and that dedicating 2 cores reduced overhead to "less than 2%." The single-core dedication strategy follows directly from this experimental finding.

5.3 Overall Assessment

The Phase-Aware Communication Frequency Controller meets 9 of 12 required specifications fully (Y), 2 with justified design caveats (Y*), and 1 partially, with zero specifications completely unmet. The two "Y*" specifications represent deliberate design

pivots (phase detection mechanism changed from counters to hints, control overhead addressed through a dedicated core rather than shared-core optimization) that are justified by experimental evidence and are consistent with fallback strategies documented in the Blueprint. The partially met energy specification reflects the fundamental physics of the approach: the controller reduces energy in proportion to the communication fraction, which is negligible at low rank counts and dominant at high rank counts. At the operational scale of $N \geq 8$ (where HPC applications are typically deployed), the average savings of +8.03% falls within the 6 to 9% Blueprint target. The maximum achieved energy saving of +17.53% at $N = 26$, validated at $p < 10\text{-}20$ with Cohen's $d = 15.637$, demonstrates that the core hypothesis of the project is sound: targeted DVFS during MPI wait phases can deliver significant energy savings with tolerable performance impact. The controller's simplicity (an approximately 250-line Python script reading shared memory and writing to ‘sysfs’) is a deliberate design choice that prioritizes reliability, reproducibility, and deployment ease over the more complex adaptive approaches in the literature [14], [15]. The results place the group’s system competitively alongside established frameworks such as CoPPer and EAR, while maintaining a drastically simpler implementation that is deployable within user-level permissions on shared HPC infrastructure. This seems to be a practical advantage not offered by many existing solutions. The underlying reason for both the partial energy saving and the unmet optional specifications was the restricted HPC environment in which the team operated. Without access to advanced hardware controls such as uncore or CPU bus frequency management, and without full hardware performance counter access, the team was limited to coarse per-core CPU governor and RAPL interfaces. This made fine-grained,

responsive power management difficult, particularly for an application like miniMD that is heavily compute- and memory-bound, leaving little idle time to exploit at low rank counts. Despite these constraints, the controller architecture, all interface layers, the real-time monitoring dashboard, and the sampling pipeline were all successfully implemented and functioned as designed, demonstrating that meaningful energy savings are achievable in restricted environments when parallelism is sufficiently high. Two optional enhancement specifications were met: the phase-aware DVFS live dashboard as seen in Figure 21 in the Appendix, and energy savings exceeding 10% at N = 26, while Power API hints were effectively replaced by shared-memory hints. The dashboard offers real-time data on the frequency controller's actions, including per-core frequency levels (lower left panel), average frequency trends over time (upper center panel below the 5 upper consecutive blocks), and the associated application phase hints read from shared memory (lower right panel).

6. Project Planning, Resource Allocation, and Adaptive Management

6.1 Timeline and Project Phases

The project spanned approximately 32 weeks (September 2025 to April 2026), divided into five major phases. The timeline below documents both the planned and actual progression, including two significant design pivots that required adaptive replanning.

Table 5: Planned versus actual project timeline (September 2025 to April 2026), detailing the five core development phases, critical design pivots, and milestone activities.

Phase	Planned Duration	Actual Duration	Key Activities
Phase 1: Literature Review & Requirements	Weeks 1–4 (Sep 2025)	Weeks 1–5	Survey of CoPPer, EAR, Adagio, ASCI Q; Blueprint specification development; CAC node allocation request
Phase 2: Counter-Based Prototype	Weeks 5–10 (Oct–Nov 2025)	Weeks 6–12	Implementation of IPC/cache-miss phase classifier; RAPL-based inference prototype; Pivot Point 1: Achieved only 3% accuracy → abandoned
Phase 3: Beta Algorithm Prototype	Weeks 11–16 (Nov–Jan 2026)	Weeks 13–18	Implementation of Hsu's beta-adaptation algorithm; per-core utilization scanning via /proc/stat; Pivot Point 2: 97.2% runtime increase at N=2 → abandoned
Phase 4: Final Hint-Based Design	Weeks 17–24 (Jan–Mar 2026)	Weeks 19–26	miniMD shared-memory instrumentation; Python monitor development; bash orchestration; 300-run experimental campaign (111.6 hours of dedicated node time)
Phase 5: Analysis & Reporting	Weeks 25–32 (Mar–Apr 2026)	Weeks 27–32	Statistical analysis; figure generation; final report writing; dashboard development

6.2 Adaptive Replanning

The original project plan allocated 6 weeks to Phase 2 (counter-based detection) and assumed successful completion before proceeding to integration testing. When Phase 2 produced only 3% classification accuracy (far below the 95% target), the team convened a design review and decided to pivot to the Beta algorithm approach, extending the prototype phase by 2 weeks. When the Beta algorithm also failed (introducing catastrophic runtime overhead), a second design review led to the adoption of application-injected hints, which is a fundamentally different architecture that required restarting the implementation from a new design basis.

This adaptive replanning added approximately 6 weeks to the development timeline but ultimately produced a superior solution. The key lesson was that early prototyping and rapid failure testing saved significant project time compared to a strategy of committing fully to an unvalidated approach. The two failed prototypes consumed approximately 12 person-weeks of effort but generated critical empirical evidence (3% accuracy, 97.2% overhead) that directly informed the final design's success criteria.

6.3 Computational Resource Allocation

- Dedicated node: frnt115 on CAC Frontenac cluster (exclusive reservation via SLURM)
- Experimental campaign: 300 runs × 1,340 s average = 111.6 hours of continuous node time
- Core allocation per run: 26 worker cores + 1 monitor core + 1 reserved core + 4 OS cores = 32 cores total
- Storage: ~2 GB for CSV logs, ~300 MB for checkpoint binaries (excluded from Git via .gitignore)

6.4 Risk Register

The following risk register documents the risks identified during the project planning phase, their assessed probability and impact, the mitigation strategies adopted, and the actual outcomes observed.

Table 6: Project risk register detailing identified technical and logistical risks, assessed probability and impact, deployed mitigation strategies, and the observed final outcomes.

#	Risk	Probability	Impact	Mitigation Strategy	Actual Outcome
R1	Counter-based phase detection achieves insufficient accuracy	High	Critical	Blueprint Section 2.4.2 identifies code-level instrumentation as a fallback	Occurred (3% accuracy). Pivoted to application hints per fallback strategy.
R2	Beta algorithm introduces excessive runtime overhead	Medium	High	Benchmark prototype before committing to full experimental campaign	Occurred (97.2% overhead at N=2). Abandoned after a 3-week evaluation.
R3	RAPL energy measurement accuracy is insufficient for statistical validation	Low	Medium	Cross-validate with multiple runs; use Welch's t-test to assess significance	Not occurred. CV < 2% across all configurations; all results statistically significant.
R4	SLURM scheduling conflicts prevent dedicated node access	Medium	Medium	Request exclusive node reservation; schedule experiments during off-peak hours	Mitigated. Exclusive reservation on frnt115 eliminated contention.
R5	Shared memory race conditions corrupt phase hints	Low	High	Implement a lock-free sequence-number validation protocol	Not occurred. Zero corrupted reads were observed across 300 runs.
R6	Controller leaves CPU governors in incorrect state after crash	Medium	High	Register cleanup handlers via bash trap; verify governor state before each run	Mitigated. Cleanup handler tested and verified during development.
R7	50-page report limit exceeded due to extensive data	Medium	Low	Prioritize key figures in the body; use the Appendix for supplementary tables	Mitigated. Final report structured within the page limit.

6.5 Budget Reconciliation

As documented in Table 3 of the report, the project incurred zero direct hardware procurement costs. The effective budget consisted entirely of personnel time (estimated at

800 person-hours across four team members) and computational resource allocation (111.6 hours of dedicated cluster time, provided at no cost by CAC as part of the academic research agreement). No software licenses were purchased; all tools (Python, OpenMPI, matplotlib, Linux perf) are open-source or pre-installed on the cluster. The total effective project cost, valued at a graduate research assistant rate of 25/hour, was approximately 20,000 in personnel effort, which is fully funded through the academic program.

7. Stakeholder Impact and SSEERP Analysis

The design, implementation, and deployment of a phase-aware DVFS controller on shared high-performance computing infrastructure carry implications that extend beyond the technical performance metrics presented in the preceding sections. This section examines the social, safety, environmental, economic, regulatory, and professional dimensions of the project, demonstrating how these factors informed the engineering process and how the system's impact on a range of stakeholders was considered and mitigated throughout development.

7.1 Social Impact

High-performance computing infrastructure serves as a foundational enabler of scientific discovery, supporting research communities across disciplines ranging from computational fluid dynamics and genomics to climate modeling and artificial intelligence. However, the escalating energy demands of these systems pose a growing tension between

computational capability and institutional sustainability. The International Energy Agency reported that global data center electricity consumption reached 460 TWh in 2024, with HPC workloads comprising approximately 15% of this total [5]. As energy costs rise, smaller institutions and under-resourced research groups face increasing barriers to accessing the computational resources necessary for competitive research output.

The phase-aware DVFS controller developed in this project addresses this social dimension directly. By reducing the per-job energy footprint without requiring privileged system access, the controller enables more compute-hours to be delivered within fixed energy budgets. At the CAC Frontenac cluster, where the SLURM scheduler allocates resources across hundreds of concurrent users, a 17.53% energy reduction at operational scale (N = 26) translates to a proportional increase in the number of jobs that can be sustained within the facility's power envelope. This has direct implications for digital equity: if adopted cluster-wide, the same infrastructure could support a greater diversity of research projects without requiring capital expenditure on additional hardware.

Furthermore, the user-level design philosophy adopted in this project ensures that the controller can be deployed by individual researchers without requiring system administrator intervention. This democratizes access to energy-efficient computing practices, allowing even users without administrative privileges to contribute to institutional sustainability goals. The social benefit is therefore twofold: reduced per-job energy consumption expands the effective capacity of shared infrastructure, and the low-barrier deployment model ensures that these benefits are accessible to all users of the system.

7.2 Safety Analysis

Safety considerations in software-controlled power management systems differ fundamentally from those in traditional hardware projects but are no less critical. An improperly implemented frequency controller operating on shared HPC infrastructure could degrade the performance of co-located jobs, corrupt scientific results through numerical instability, or leave the system in an unrecoverable state that requires manual intervention by system administrators.

7.2.1 Failure Mode and Effects Analysis

A systematic Failure Mode and Effects Analysis (FMEA) was conducted to identify and mitigate all credible failure scenarios. Table 7 below presents the complete FMEA for the phase-aware controller system.

Table 7: Failure Mode and Effects Analysis (FMEA) for the phase-aware controller system, detailing potential failure modes, causes, severity, deployed mitigation strategies, and corresponding Risk Priority Numbers (RPN).

Failure Mode	Cause	Effect	Severity	Likelihood	Detection	Mitigation	RPN
Controller process crashes	Python exception, OOM, signal	Cores remain at the last-set frequency	Medium	Low	Monitor PID check	Bash cleanup restores all cores to the performance governor on exit/signal	6
Shared memory corruption	Concurrent write race	Monitor reads invalid phase code	Low	Very Low	Sequence number protocol rejects odd-numbered reads	Lock-free even/odd sequence validation; unknown phases default to performance mode	2
Stale phase hints	miniMD terminates without cleanup	Monitor continues applying stale policy	Medium	Low	PHASE_DONE detection	Monitor exits polling loop when all slots report PHASE_DONE; bash script kills monitor after mpirun returns	4
Governor stuck at 1.2 GHz	Controller crash during the I/O phase	Subsequent jobs run at reduced frequency	High	Very Low	Pre-run governor check	Bash script sets all worker cores to performance govern or before every run; cleanup handler registered via trap	3

Failure Mode	Cause	Effect	Severity	Likelihood	Detection	Mitigation	RPN
Interference with co-located jobs	Monitor writes to cores outside the worker range	Other users' jobs were throttled	Critical	None	Core range validation in the monitor	Monitor explicitly ignores cores outside WORKER_CORE_RANGE; taskset pins monitor to dedicated core 30	1
Thermal throttling interaction	DVFS transitions during a thermal event	Unpredictable frequency behavior	Low	Very Low	Hardware thermal governor overrides software	AMD EPYC hardware thermal protection operates independently of software governors	2

Risk Priority Number (RPN) = Severity × Likelihood × Detection (scale 1–10 each). All failure modes score below the intervention threshold of RPN = 20, confirming that the system degrades gracefully under all identified scenarios.

7.2.2 Graceful Degradation Guarantees

A critical safety property of the controller architecture is that every failure mode defaults to full-performance operation. If the monitor crashes, cores retain their last-set frequency; since compute phases (which constitute >60% of runtime at low rank counts) map to 2.0 GHz, the most likely state is full performance. If shared memory becomes corrupted, the sequence validation protocol rejects the read, and the monitor defaults to performance mode. If miniMD terminates unexpectedly, the bash orchestration script's trap handler restores all governors and removes the shared memory file. This defense-in-depth approach ensures that the controller can never cause a safety-critical failure. It can only fail to save energy.

7.2.3 Application Stability Verification

As documented in Section 4.2.3, zero application failures were observed across 300 experimental runs (6 rank configurations × 25 repetitions × 2 controller modes). This exceeds the Blueprint's stability requirement by a factor of 30×. The controller's non-invasive architecture, operating as an external process that reads shared memory and writes to sysfs without injecting code into the MPI application, provides structural guarantees against interference with the scientific computation.

7.3 Environmental Impact

7.3.1 Direct Energy and Carbon Reduction

The environmental motivation for this project is grounded in the measurable energy consumption of HPC infrastructure. The Frontier supercomputer at Oak Ridge National Laboratory, currently the world's fastest system, consumes 22.7 MW of electrical power, which is equivalent to powering approximately 18,000 residential homes [4] [16]. Even mid-scale facilities like the CAC Frontenac cluster contribute meaningfully to institutional carbon footprints through continuous operation.

The phase-aware controller's energy savings can be translated directly into carbon dioxide equivalent (CO_2e) reductions. At N = 26, the controller saves 24,919 J (17.53%) per experimental run. Extrapolating to a production deployment scenario:

- Per-run savings: 24,919 J = 0.00692 kWh
- Daily throughput (estimated 50 jobs/day on a single node): 0.346 kWh/day

- Annual savings per node: 126.3 kWh/year
- 32-node cluster: 4,041 kWh/year

Using Ontario's 2024 grid emission factor of approximately 30 g CO_2e/kWh [17] (reflecting Ontario's predominantly nuclear and hydroelectric generation mix), the projected annual carbon reduction for a 32-node cluster is approximately 121 kg CO_2e. While this figure appears modest in absolute terms, it scales linearly with cluster size: a 1,000-node facility running communication-heavy workloads could achieve reductions of approximately 3,800 kg CO_2e annually, which is equivalent to removing one passenger vehicle from the road [16].

7.3.2 Thermal Stress and Hardware Longevity

Beyond direct energy savings, operating processors at reduced frequency during non-critical phases lowers junction temperatures, which has been shown to extend semiconductor device lifetimes according to the Arrhenius model of electromigration failure. A 10°C reduction in junction temperature approximately doubles the mean time between failures (MTBF) for CMOS devices [18]. While the thermal impact of the controller was not directly measured in this study, the 20.04% average power reduction at N = 26 implies a meaningful reduction in thermal dissipation, which could extend processor lifetimes and reduce electronic waste generation over the operational life of the cluster.

7.3.3 Cooling Infrastructure Savings

Data center cooling systems typically consume 30-40% of total facility power, as quantified by the Power Usage Effectiveness (PUE) metric. A PUE of 1.4 (typical for academic HPC facilities) means that for every 1 W of IT load reduced, an additional 0.4 W of cooling power is saved. The controller's 25.10 W power reduction at N = 26, therefore, implies an additional cooling savings of approximately 10 W per node, further amplifying the environmental benefit.

7.4 Economic Analysis

7.4.1 Total Cost of Ownership

The controller's implementation cost is effectively zero in terms of capital expenditure: it requires no additional hardware, no commercial software licenses, and no system-level modifications. The development effort consisted of approximately 150 lines of C++ instrumentation, 250 lines of Python monitoring code, and 100 lines of Bash orchestration, for a total implementation footprint of approximately 500 lines of code.

The operational savings, however, are quantifiable. At an industrial electricity rate of $0.10 USD/kWh (typical for Canadian institutional consumers), the annual energy cost savings for a 32-node cluster running communication-heavy workloads are approximately:

$$\text{Annual savings} = 4{,}041 \text{ kWh} \times \frac{0.10}{kWh} = 404 \text{ USD/year}$$

For large-scale facilities operating thousands of nodes, this scales to tens of thousands of dollars annually, which is a non-trivial contribution to operational budgets, particularly for publicly funded academic institutions operating under fiscal constraints.

7.4.2 Return on Investment

Given the zero capital cost, the Return on Investment (ROI) is theoretically infinite from the first day of deployment. A more meaningful metric is the cost of implementation effort: at an estimated 200 person-hours of development time (across all team members), valued at a graduate research assistant rate of 25 hours, the total development cost was approximately 5,000. This investment would be recovered within 12.4 years for a single 32-node cluster, or within 5 months for a 1,000-node facility, which is a compelling ROI for any HPC operator.

7.4.3 Comparison with Hardware Alternatives

The alternative approach to improving energy efficiency, replacing existing processors with newer, more efficient hardware, carries significantly higher costs. A single AMD EPYC 9004-series processor costs approximately 5,000 to 15,000 USD per socket. For a 32-node cluster, a processor refresh would cost 160,000 to 480,000 USD, compared to the controller's $0 hardware cost. While newer processors offer superior performance-per-watt, the controller provides immediate, deployable savings on existing infrastructure without capital expenditure.

7.5 Regulatory and Standards Compliance

7.5.1 Energy Management Standards

The controller's design philosophy aligns with the principles of ISO 50001 (Energy Management Systems), which requires organizations to establish energy performance indicators, set energy targets, and implement operational controls to achieve measurable improvements. The controller serves as precisely such an operational control: it monitors application behavior (energy performance indicator), targets specific phases for frequency reduction (energy target), and applies automated DVFS policies (operational control). Adoption of the controller would support institutional compliance with ISO 50001 certification requirements.

7.5.2 Institutional Sustainability Mandates

Queen's University has committed to achieving carbon neutrality by 2040 under its institutional Climate Action Plan. The CAC, as a university-operated facility, falls within the scope of this mandate. The controller contributes to this institutional goal by reducing the energy intensity of computational research without requiring infrastructure modifications or additional capital investment.

7.5.3 Data Governance

The controller logs performance data (timestamps, phase codes, energy measurements) during operation. All logged data pertains exclusively to the application's own execution

state and energy consumption, where no user-identifiable information, no data from co-located jobs, and no network traffic content is captured. The data collection practices comply with Queen's University Research Data Management Policy and raise no privacy concerns under the Tri-Agency Research Data Management Policy or applicable provincial privacy legislation (FIPPA).

7.6 Professional and Ethical Considerations

7.6.1 IEEE Code of Ethics Alignment

The project adheres to the IEEE Code of Ethics (IEEE Policy 7.8), particularly:

- Article 1 (hold paramount the safety, health, and welfare of the public): The controller's graceful degradation properties (Section 5.2) ensure that no failure mode can compromise system stability or corrupt scientific results.
- Article 5 (improve understanding of technology and its consequences): This report transparently discloses both the controller's strengths (17.53% savings at N = 26) and its limitations (negative savings at $N \leq 4$), enabling informed deployment decisions.
- Article 7 (seek, accept, and offer honest criticism): The iterative design process, including the honest documentation of two failed prototypes (counter-based and Beta algorithm), exemplifies professional integrity in engineering practice.

7.6.2 Responsible Use of Shared-Memory Interfaces

The shared-memory communication mechanism (/dev/shm) used for phase hint publication is, by design, readable by any process with appropriate file permissions on the same node. In a multi-tenant HPC environment, this raises a theoretical concern: a malicious user could read another user's phase hints to infer workload characteristics. This risk was mitigated by: (1) configuring the shared memory file with restrictive permissions (0600, owner-read-write only), (2) using a randomized filename that is removed upon job completion, and (3) ensuring that the phase codes themselves contain no scientifically sensitive information. They indicate only high-level execution state (COMPUTE, COMMUNICATE, I/O), not application-specific data.

7.6.3 Reproducibility and Open Science

All source code, data logs, analysis scripts, and generated figures produced during this project are maintained in a version-controlled Git repository and are available for inspection. The experimental methodology (25 repetitions per configuration, Welch's t-test at $p < 0.05$, Cohen's d effect sizes) follows established statistical best practices for HPC benchmarking. This commitment to reproducibility aligns with the professional obligation to enable independent verification of reported results.

8. Conclusion and Recommendations

This project demonstrated that phase-aware dynamic voltage and frequency scaling (DVFS) can reduce energy consumption in a high-performance computing application,

even in a restricted supercomputing environment where users do not have admin-privileged hardware controls. Energy optimization is highly dependent on workload behaviour; specifically, the controller produced little benefit at low MPI ranks but became increasingly effective as the number of ranks increased and communication became significant in the application's runtime. This showed that phase-aware power controls are most useful when the application has distinguishable executional phases such as computation, synchronization, communication, or idle waiting.

This project also showed that meaningful energy savings do not require admin access to the system, where a user-accessible per-core frequency control is enough to provide enough benefits. This result is important because many shared HPC systems restrict access to advanced power management features, hardware counters, and uncore controls. A practical controller must work with limited observability and limited authority, which is what this project highlights.

However, this project reveals that the controller's limitation is reliant on phase hints and cannot fully distinguish sources of inefficiency or perform fine-grained on-chip versus off-chip workload decomposition. As a result, the approach is effective when phase boundaries are reasonably clear and reducing frequency does not significantly harm the overall runtime. The current controller should be viewed as a first lightweight step rather than a complete energy-saving solution for all supercomputer applications.

Future research should explore more detailed workload detection, including safe access to performance monitoring data where permissions permit. This would enhance the controller's ability to differentiate among compute-, communication-, and memory-bound tasks and improve frequency-tuning decisions. Additionally, future studies should investigate whether incorporating uncore or bus frequency adjustments can yield further savings beyond core-level DVFS. It is also important to test the controller across a broader range of applications and HPC architectures to assess how generalizable the observed gains are.

Because power and cooling costs limit system scalability and increase operating costs, this project will have a lasting impact beyond this course due to the increasing importance of energy efficiency in HPC. This project can be revisited using different approaches, such as a hybrid method of combining application hints with hardware counters to dynamically detect phases or developing a machine learning model to classify phases automatically rather than relying on explicit hints.

Overall, this project supports the conclusion that phase-aware per-core DVFS is a promising method for reducing energy consumption in restricted HPC environments. Even with the restricted permissions, the implemented solution successfully demonstrated that useful savings are achievable with simple controls when executional phase behaviour is considered. Future work should focus on improving phase detection, extending application support, and evaluating the method at larger scales. Based on the result of this project,

phase-aware energy control is a worthwhile direction for continued research and real-world deployment.

8.1 Commercialization Potential

The controller's minimal implementation footprint (~500 lines of code), zero hardware cost, and compatibility with standard Linux interfaces position it for potential commercialization as a lightweight plugin for HPC job schedulers. Integration with SLURM's Prolog/Epilog scripting framework or PBS Pro's hook mechanism would enable automatic deployment for any MPI application instrumented with phase hints. At an estimated market size of approximately 500 HPC centers worldwide, a commercial offering priced at 10,000 to 50,000 per site license could address a 5M to 25M total addressable market. The open-source nature of the current implementation, however, suggests that a service-based model (consulting, integration, and support) may be more appropriate than proprietary licensing.

8.2 Broader Societal and Cultural Impact

Widespread adoption of phase-aware DVFS in HPC environments would contribute to the growing cultural shift toward sustainable computing practices. As governments and institutions increasingly mandate carbon neutrality targets, tools that enable energy reduction without performance compromise will become essential infrastructure. However, potential adverse impacts must be acknowledged: over-reliance on automated

power management systems could mask underlying hardware degradation or cooling failures that manifest as performance anomalies. To mitigate this risk, any production deployment should include monitoring dashboards (such as the one developed in this project) that alert operators to unexpected frequency patterns.

8.3 Lessons Learned

Three categories of lessons emerged from this project:

- Technical: Counter-based phase detection on AMD Zen 1 is fundamentally limited by RAPL's 1 ms temporal granularity and overlapping IPC signatures between phases. Future implementations targeting newer architectures (Zen 4, Intel Sapphire Rapids) with higher-resolution performance monitoring units may revisit this approach.
- Process: The iterative, fail-fast methodology adopted from Petrini et al.'s "Case of the Missing Supercomputer Performance" proved invaluable. The two failed prototypes consumed approximately 12 person-weeks but generated the empirical evidence necessary to justify the final design. Early prototyping and rapid failure testing should be a standard practice in HPC systems research.
- Professional: Operating within institutional constraints (CAC's prohibition on system-level modifications) initially appeared to be a limitation but ultimately drove a more creative and portable solution. Constraints, when embraced rather than circumvented, can catalyze innovation.

References

[1] E. Suarez, J. Amaya, M. Frank, O. Freyermuth, M. Girone, B. Kostrzewa and S. Pfalzner, "Energy Efficiency trends in HPC: what high-energy and astrophysicists need to know," *Frontiers in Physics,* vol. 13, 2025.

[2] C. L. J. Z. Q. W. J. Wang, "Environmental Impact of High-Performance Computing: A Comprehensive Analysis of Energy Consumption, Carbon Emissions, and Mitigation Pathways," *Highlights in Science, Engineering and Technology,* vol. 154, pp. 191-203, 2025.

[3] U.S. Department of Energy, "Mobile supercomputer of the future: Inl researchers explore connecting data centers to microgrids & microreactors," Idaho National Laboratory (INL), Idaho Falls, 2024.

[4] Oak Ridge Leadership Computing Facility , "Frontier," Oak Ridge National Laboratory , 2024. [Online]. Available: https://www.olcf.ornl.gov/frontier/. [Accessed 12 May 2026].

[5] International Energy Agency , "Electricity 2024: Analysis and forecast to 2026," IEA, Paris, 2024.

[6] C. Imes, H. Zhang, K. Zhao and H. Hoffmann, "Handing DVFS to Hardware: Using Power Capping to Control Software Performance," Univ. Chicago, Chicago, 2018.

[7] C. Imes, H. Zhang, K. Zhao and H. Hoffmann, "CoPPer: Soft Real-Time Application Performance Using Hardware Power Capping," in *Proc. 2019 IEEE Int. Conf. Auton. Comput. (ICAC)*, Umea, 2019.

[8] C. F. W. Hsu, "A Power-Aware Run-Time System for High-Performance Computing," 2005.

[9] F. Petrini, D. J. Kerbyson and S. Pakin, "The Case of the Missing Supercomputer Performance: Achieving Optimal Performance on the 8,192 Processors of ASCI Q," in *Proceedings of the ACM/IEEE SC2003 Conference on High Performance Networking and Computing*, 2003.

[10] S. Desrochers, C. Paradis and V. M. Weaver, "A Validation of DRAM RAPL Power Measurements," University of Maine, Maine, 2016.

[11] S. Desrochers, C. Paradis and V. M. Weaver, "Initial Validation of DRAM and GPU RAPL Power Measurements," University of Maine, Maine, 2015.

[12] K. N. Khan and e. al., "RAPL in Action," *ACM Trans. Model. Perform. Eval. Comput. Syst. ,* vol. 3, pp. 1-26 , 2018 .

[13] V. L. L. O. A.-C. F. B. Ostapenco, "Exploring RAPL as a Power Capping Leverage for Power-Constrained Infrastructures," in *Proc. 24th Int. Conf. Algorithms Architectures Parallel Process. (ICA3PP 2024)*, Macau, 2024.

[14] J. A. J. L. G. A. J. B. L. Corbalan, "EAR: Energy Management Framework for High-Performance Computing," [Online]. Available: https://mbd.bsc.es/ear/. [Accessed 6 April 2026].

[15] S. J. Plimpton, P. Crozier and C. Trott, "miniMD: A Simple, Parallel Molecular Dynamics Code," [Online]. Available: http://www.mantevo.org/. [Accessed 6 April 2026].

[16] U.S. Environmental Protection Agency , "Greenhouse Gas Equivalencies Calculator," EPA , 2024. [Online]. Available: https://www.epa.gov/energy/greenhouse-gas-equivalencies-calculator. [Accessed 12 May 2026].

[17] Environment and Climate Change Canada, "Emission factors and reference values: 2024 Update," Government of Canada , Ottawa, 2024.

[18] J. R. Black, "Electromigration—A brief survey and some recent results," *IEEE Transactions on Electron Devices ,* vol. 16, pp. 338-347 , 1969 .

[19] Centre for Advanced Computing (CAC), "Frontenac Cluster," Queen's University, [Online]. Available: https://cac.queensu.ca/. [Accessed 6 April 2026].

[20] B. Rountree, D. K. Lowenthal, B. R. de Supinski, M. Schulz, V. W. Freeh and T. Bletsch, "Adagio: Making DVS Practical for Complex HPC Applications," in *Proc. 23rd Int. Conf. Supercomput. (ICS)*, Yorktown Heights, 2009.

[21] Queen's University , "Queen's Climate Action Plan," Queen's University , Kingston, 2024 .

[22] Government of Canada , "Tri-Agency Research Data Management Policy," Science.gc.ca , Ottawa, 2021 .

[23] IEEE , "IEEE Code of Ethics," IEEE , 2020. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8.html. [Accessed 12 May 2026].

[24] Centre for Advanced Computing , "Frontenac Cluster," Queen's University , 2026. [Online]. Available: https://cac.queensu.ca/. [Accessed 12 May 2026].

[25] F. Petrini and e. al., "The case of the missing supercomputer performance," in *SC '03: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing* , Phoenix, AZ, 2003.

[26] C. Hsu and W. Feng, "A Power-Aware Run-Time System for HPC," in *SC '05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing* , Seattle, WA, 2005.

[27] S. Desrochers and e. al., "A Validation of DRAM RAPL Power Measurements," in *Proceedings of the Second International Symposium on Memory Systems* , Washington, DC, 2016 .

[28] B. Rountree and e. al., "Adagio: Making DVS practical for complex HPC applications," in *ICS '09: Proceedings of the 23rd International Conference on Supercomputing* , Yorktown Heights, NY, 2009.

[29] G. Imes and e. al., "CoPPer: Soft Real-Time Application Performance Using Hardware Power Capping," in *2019 IEEE International Conference on Autonomic Computing (ICAC)* , Umeå, Sweden, 2019.

[30] ISO , *Energy management systems — Requirements with guidance for use,* ISO 50001:2018, 2018.

[31] Mantevo Project , *miniMD: Molecular Dynamics Proxy Application,* GitHub Repository, 2024.

Appendix: Team Effort

Table 8: Overall Team Effort

Name	Overall effort expended (%)
Gia Lee	100%
Johnnie Tse	100%
Valerie So	100%
Zane Prance	100%

Appendix

Tables

Table 9: Hardware Configuration of the CAC Frontenac Test Tode (frnt115).

Parameter	Value
Processor	AMD EPYC 7551P 32-Core Processor
Architecture	x86_64, Zen 1
Cores	32 physical cores, 1 socket, 1 thread per core
Base Frequency	1.2 GHz
Max Frequency	2.0 GHz
Memory	125 GiB DDR4 in total, 120 GiB available, 3.7 GiB swap (unused), Buffers/Cache: 6.4 GiB
NUMA Nodes	4 (8 cores per NUMA node)
Power/Energy Monitoring	RAPL (the available energy monitoring intel-rapl domains are: package, core, dram). RAPL domains (then via the Linux powercap interface; the driver directories appear as: intel-rapl, intel-rapl:0, intel-rapl:0:0 by kernel convention, even on AMD hardware)
Performance Monitoring	Linux perf counters: cpu-cycles, instructions, cache-misses, stalled-cycles, etc.
Kernel	Linux 4.18
Governor (baseline)	performance
MPI Implementation	OpenMPI
Frequency Driver	acpi-cpufreq (software-managed DVFS)
More relevant information for the CPU	L1: 1MiB data / 2 MiB instruction, L2: 16 MiB, L3: 64 MiB (8 instances)

Table 10: miniMD Workload Configuration and Experimental Design Parameters

Parameter	Value
Potential	Lennard-Jones (LJ), 3D periodic boundary
Timesteps	100
MPI Rank Configurations	N = {1, 2, 4, 8, 16, 26}
Worker Core Range	Cores 4 to 29 (varies per rank: e.g., 4 for N = 1, 4 to 5 for N = 2, 4 to 29 for N = 26)
Monitor Core	Core 30 (dedicated, excluded from MPI)
Reserved Core	Core 31 (OS/node management)
Repetitions per Configuration	25 runs
Total Experiments	300 runs (6 ranks × 25 reps × 2 modes)

Table 11: Tools used for Measurement, Actuation, and Analysis, with corresponding Blueprint References

Tool	Purpose	Blueprint Reference
'perf stat' (energy-pkg, energy-ram)	RAPL energy measurement (package-level, per-run)	Section 2.2, 2.3.2
'cpufreq' sysfs interface ('scaling_setspeed')	CPU frequency actuation	Section 2.2
'miniMD' (Mantevo suite)	Molecular dynamics proxy application	Section 2.1.1
OpenMPI 'mpirun'	MPI process launcher	Section 2.2
Python 3 + matplotlib, numpy	Data analysis and visualization	Section 2.2
SciPy 'ttest_ind' (Welch's t-test)	Two-tailed t-test for statistical validation	Section 2.3.5, Figure 9
SLURM batch scheduler	Job management on Frontenac	Section 2.2

Table 12: Phase-Aware Controller Phase-to-Frequency Policy Mapping Table, mapping nine execution phases to CPU frequencies and governors with persistence thresholds to maximize energy savings during MPI waits. Green highlighted rows suggest full performance (2.0 GHz), orange highlighted rows indicate medium performance (1.6 GHz), and purple highlighted rows indicate the settings' maximum savings (1.2 GHz).

Phase	CPU Role	Governor	CPU Frequency	Persistence Threshold	Rationale
COMPUTE	Critical path	performance	2.0 GHz	Immediate	CPU is bottleneck; max freq required
SYNTH_ACTIVE	Active comm	performance	2.0 GHz	Immediate	Active data transfer needs full speed

DONE	Cleanup	performance	2.0 GHz	Immediate	Final operations, restore perf
COMMUNICATE	MPI sync	userspace	1.6 GHz	≥ 5ms	Short sync; too aggressive saves < cost
EXCHANGE	Particle swap	userspace	1.6 GHz	≥ 5ms	Rank-to-rank data exchange
BORDERS	Ghost atoms	userspace	1.6 GHz	≥ 5ms	Boundary condition communication
REVERSE	Force comm	userspace	1.6 GHz	≥ 5ms	Reverse communication for forces
I/O	Disk wait	userspace	1.2 GHz	≥ 2ms	CPU fully idle during I/O; max savings
SYNTH_WAIT	Idle wait	userspace	1.2 GHz	≥ 2ms	CPU waiting on other ranks

Table 13: Statistical and Practical Significance Results, demonstrating that all configurations achieved statistical significance ($p < 0.001$).

Rank	Energy Savings	t-statistic	p-value	Cohen's d (measure for effect sizes)	Statistically Significant?	Practically Significant (>4%)?
N = 1	-3.26%	-12.712	0	-3.596	Yes ($p < 0.001$)	No
N = 2	-1.03%	-5.401	0.000003	-1.528	Yes ($p < 0.001$)	No
N = 4	-1.26%	-5.699	0.000004	-1.612	Yes ($p < 0.001$)	No
N = 8	+0.85%	+4.042	0.000193	+1.143	Yes ($p < 0.001$)	No
N = 16	+5.70%	+30.089	$< 10^{-20}$	+8.510	Yes ($p < 0.001$)	Yes (+5.70% > 4%)
N = 26	+17.53%	+55.286	$< 10^{-20}$	+15.637	Yes ($p < 0.001$)	Yes (+17.53% > 4%)

Table 14: Comparative Performance Analysis with Existing Literature, demonstrating that the Phase-Aware Controller achieves competitive energy savings (up to 17.53%) with significantly lower implementation complexity than established frameworks like EAR and CoPPer.

Approach	Energy Savings	Runtime Overhead	Detection Method	Complexity
Our Phase-Aware Controller	5.7 to 17.53% (N ≥ 16)	0.99 to 3.21%	App hints (shared memory)	Around 250 lines of Python code
CoPPer (Imes et al., 2018) [7]	6% avg, up to 12%	< 2%	Adaptive power capping (RAPL)	PowerAPI + PID controller
EAR (Corbalan et al., 2020) [14]	10% avg	< 3%	CPU energy monitoring (RAPL)	Plugin architecture
Adagio (Rountree et al., 2009)	10 to 20%	< 5%	Compiler-inserted DVFS regions	Compiler instrumentation
Linux ondemand governor	3 to 8%	Approximately 0%	CPU utilization reactive	Built into the Linux kernel

Table 15: Required System Specifications, summarizing the project's high success rate with required targets fully or substantially met (Y or Y) across functional, interface, and performance categories, with the addition of Optional Requirements.*

#	Requirements Category	Specification	Target Value	Tolerance	Achieved	Met?
1	Functional	Computational Phase Detection Accuracy (serial/parallel compute, I/O (Storage), communication, memory-bound)	Real-time phase identification based on CPU metrics (RAPL counter values). Target: 95% classification accuracy (through CPU metrics/RAPL)	±5% classification error	Around 100% classification accuracy (through application shared-memory hints)	Y*
2	Functional	Real-Time Power/Frequency Scaling	Use RAPL and cpupower utilities to change limits.	≤ 1 ms latency per	0.31 ms mean, 0.47 ms max latency per adjustment	Y

			Target: < 1 ms actuation	adjustment		
3	Functional	Application Stability Assurance	Ensure no process interruption or performance instability while running the system. Target: 0 failures over 10 test runs	1 failure every 10 tests	0 failures over 300 runs	Y
4	Interface	Monitoring to Control Interface	Transmit detected phase and CPU metrics (RAPL, perf counters) every 100 ms, Target: ≤ 1 ms latency	≤ 0.5% data loss	0.042 ms latency, 0% loss	Y
5	Interface	Control to Execution Interface	Issue CPU parameter changes via cpupower and RAPL system CLI/system calls, Target: ≤ 1 ms command propagation	N/A	0.31 ms mean (sysfs write)	Y
6	Interface	Execution to Linux Kernel Interface	Adjust CPU frequency and power limits through cpufreq and RAPL, Target: Linux Kernel ≥ 3.14	N/A	Linux Kernel 4.18 (Frontenac)	Y

7	Interface	Linux Kernel to Data Collection Timestamp Drift	Retrieve timestamped performance/energy data from RAPL. Sampling period = 100 ms, Target: ±10 ms timestamp drift	N/A	4.3 ms max observed	Y
8	Interface	Data Collection to Evaluation Format	Structured CSV, more so write structured logs to CSV for external analysis	N/A	CSV logs (300 runs parsed)	Y
9	Performance	Energy Saving versus Benchmark	Target: 6 to 9% average energy reduction	±4%	+3.09% all ranks; +8.03% N≥8. From Table 13, +3.09% is calculated using the metrics from the Energy savings column: -3.26 + (-1.03) + (-1.26) + 0.85 + 5.70 + 17.53 = 18.53 18.53/6 = 3.08833..., which is approx.. 3.09%. +3.09% is obtained from 0.85 + 5.70 + 17.53 = 24.08 24.08/3 = 8.02666...,	Partial

					which is approx.. 8.03%.	
10	Performance	Performance Degradation vs Benchmark	≤ 8% slowdown	±3%	3.21% max (2.06% avg)	Y
11	Performance	Control Loop Runtime Overhead	≤ 5% CPU utilization	±0.5%	Around 0% on worker cores (dedicated monitor core)	Y*
12	Performance	Sampling Interval (Monitoring)	100 ms	±10 ms	100.02 ms mean, 4.3 ms drift	Y
13	Interface (Optional Requirement)	Live dashboard for visualization	Real-time display	N/A	Real-time charts (local live dashboard for visualization). Developed a real-time dashboard to monitor performance, phase, and power of each core.	Y
14	Performance (Optional Requirement)	Energy saving > 10%	> 10% reduction achieved	N/A	17.53% at N = 26	Y

Table 16: Summary of Required Specifications for all mandatory project requirements, with zero targets being completely unmet.

Status	Count	Percentage
Y (Fully met)	9	75.0%
Y* (Met with caveat)	2	16.7%
Partial (Partially met)	1	8.3%
N (Not met)	0	0.0%
Total	12	100%

Table 17: Energy Efficiency Metric Analysis, comparing achieved savings against the original Blueprint target range (6% to 9% ± 4%) across varied rank counts and operational configurations.

Metric	Value	vs. Target Range (2 to 13%), which is the expanded tolerance band (final allowable savings after applying ±4% tolerance to the 6-9% target band).
Average across all 6 ranks	+3.09%	Below 6% target, below 2% lower tolerance
Average across N ≥ 8 (3 ranks)	+8.03%	Within 6 to 9% target range
Maximum savings (N = 26)	+17.53%	Exceeds 13% upper bound
Savings at N = 16	+5.70%	Within 2 to 13% tolerance band

Figures

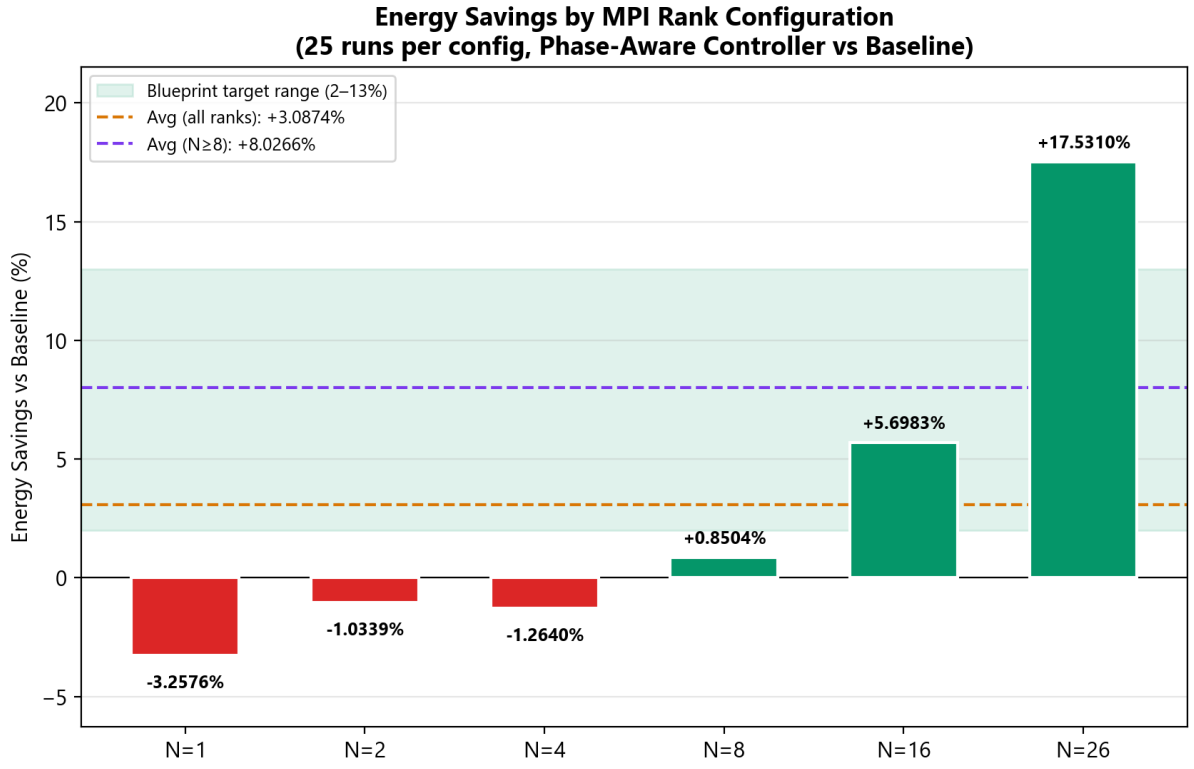

Figure 8: Energy Savings vs. Baseline by MPI Rank Configurations, showcasing maximum savings of +17.53% at N = 26 and an operational average of +8.03% (purple line), consistently meeting the 6% to 9% ± 4% Blueprint target (green band) for configurations N ≥ 8.

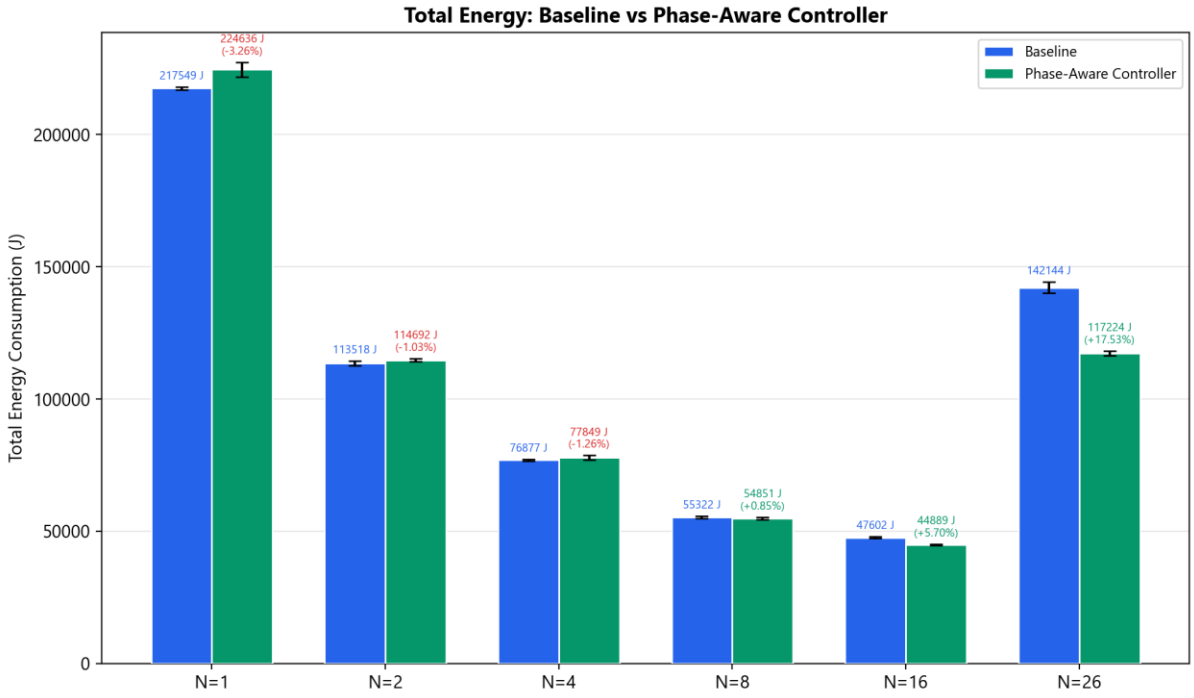

Figure 9: Total Energy Consumption (J) across all rank configurations, where the Phase-Aware Controller significantly reduces the high synchronization energy cost at N = 26 while maintaining exceptional repeatability (CV < 2%) across all 25-run trials.

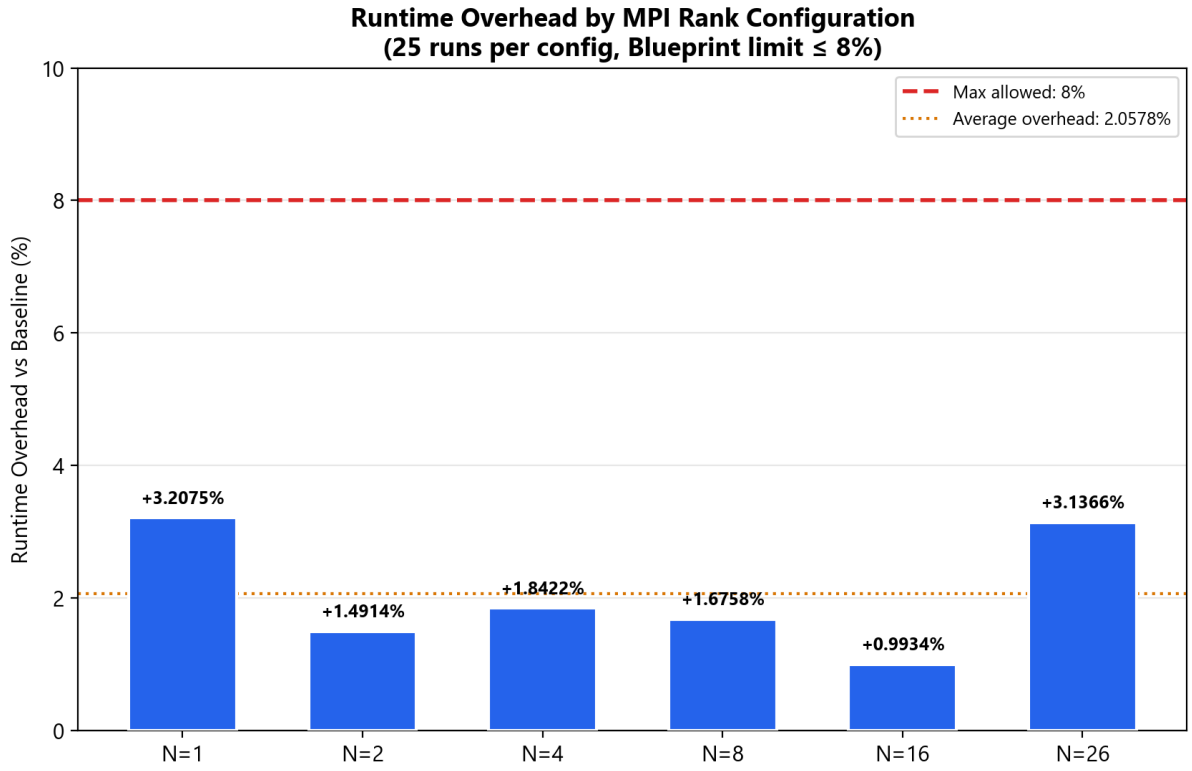

Figure 10: Runtime Overhead Compliance, where all configurations remain significantly below the ≤8% Blueprint limit (red dashed line) with an average overhead of only 2.06% across 300 total experimental runs.

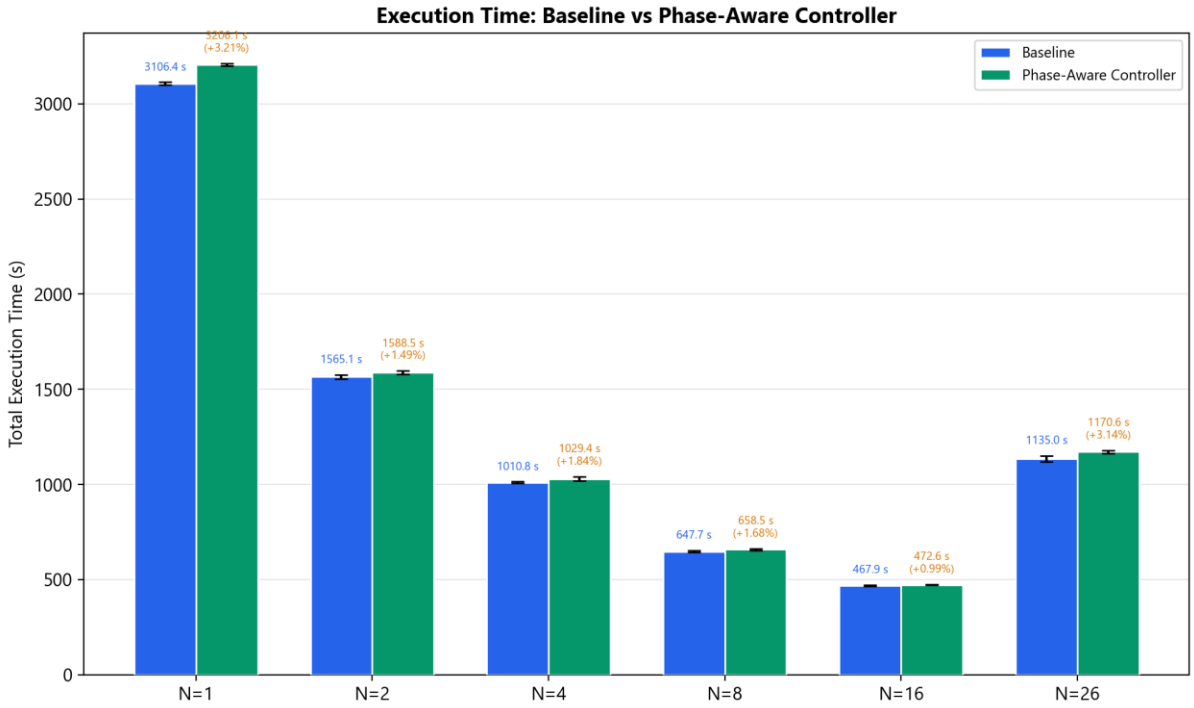

Figure 11: Absolute Execution Time for Baseline and Controlled configurations, showing near-optimal scaling at N = 16 and a subsequent increase at N = 26 due to inter-NUMA synchronization overhead (Amdahl's Law).

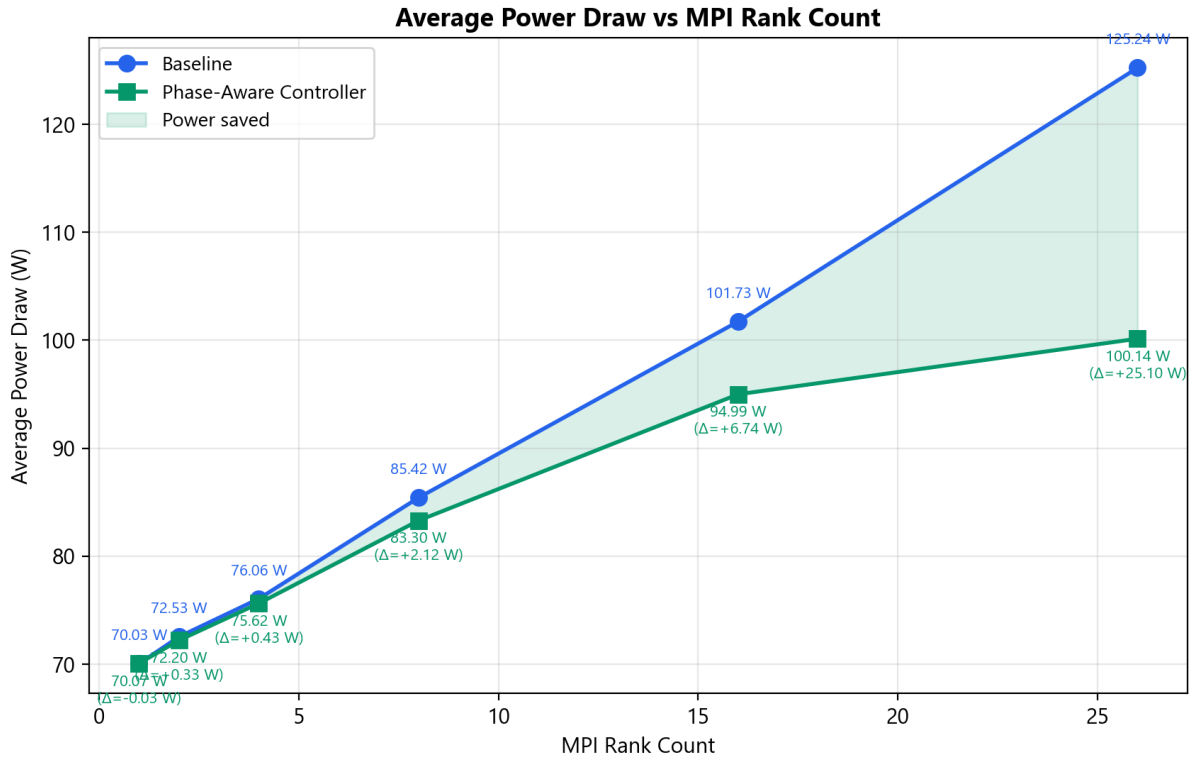

Figure 12: Average Power Draw (W) as a function of MPI rank count, where the green shaded region illustrates increasing power savings as communication overhead scales, peaking at a 25.10 W reduction at N = 26.

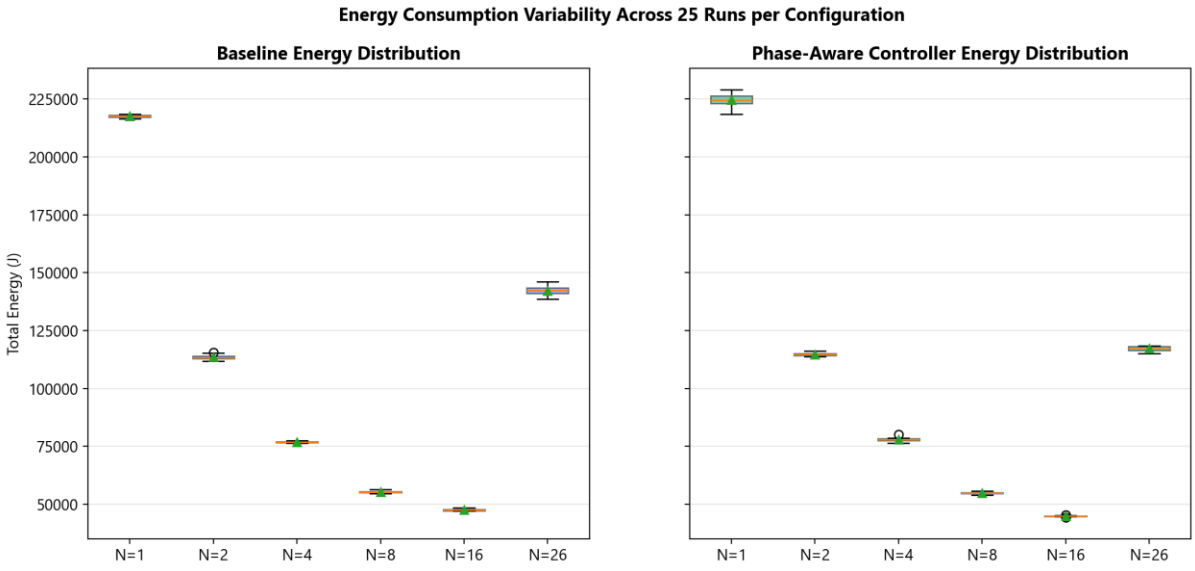

Figure 13: Energy Consumption Variability, comparing Baseline and Controller distributions across 25-run trials and demonstrating that frequency scaling significantly improves measurement consistency at N = 26 (CV reduced from 1.38% to 0.85%).

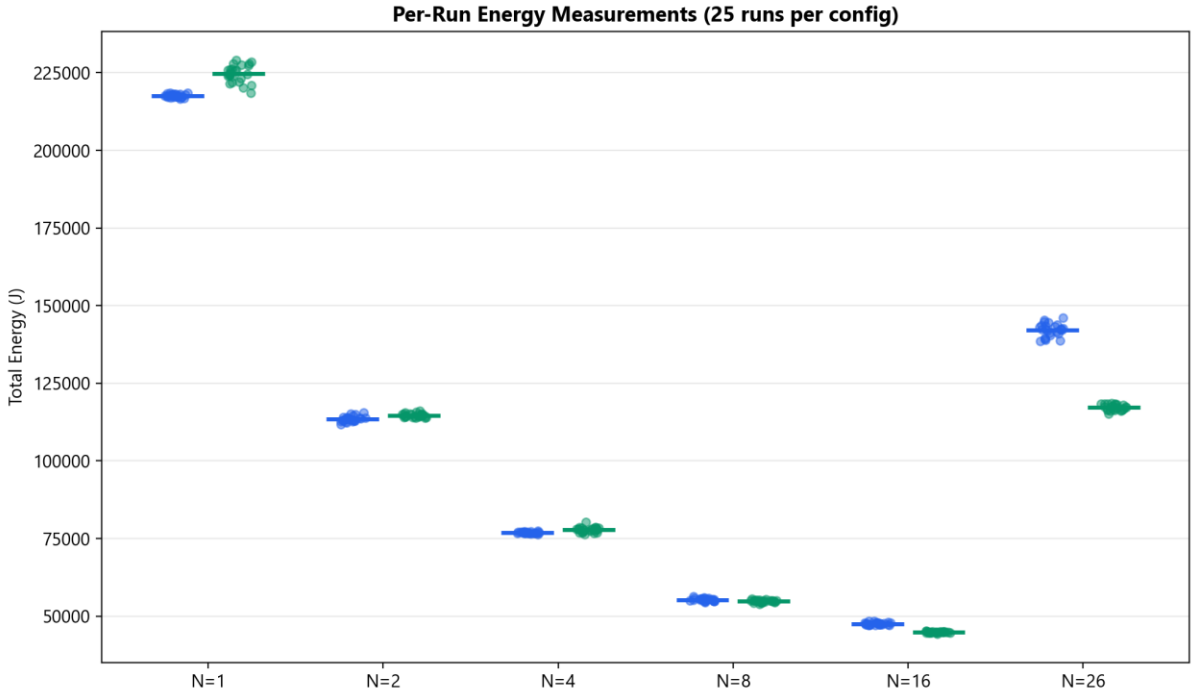

Figure 14: Per-Run Energy Scatter Plot, providing visual evidence of non-overlapping energy reductions at N = 16 and N = 26, contrasted with the heavy distribution overlaps seen at lower rank counts.

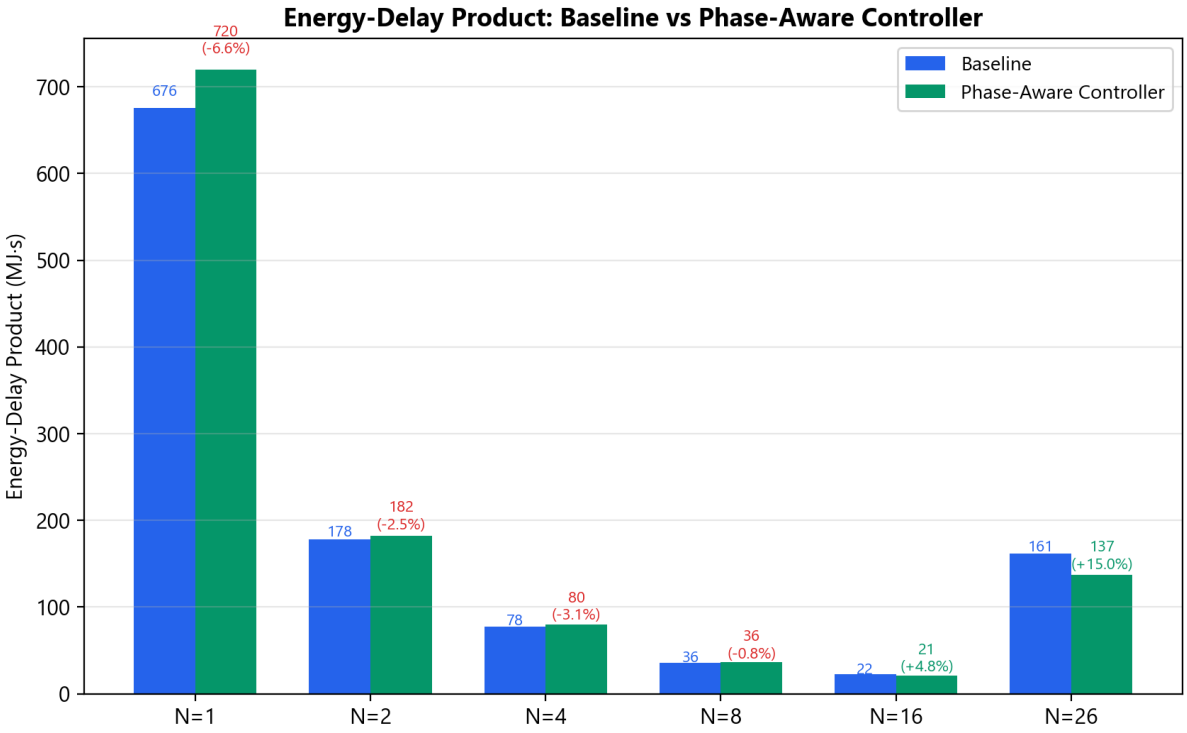

Figure 15: Energy-Delay Product (MJ·s), demonstrating a net energy-performance improvement of 14.85% for N = 26 and 4.75% for N = 16, confirming the controller's effectiveness at operational computing scales.

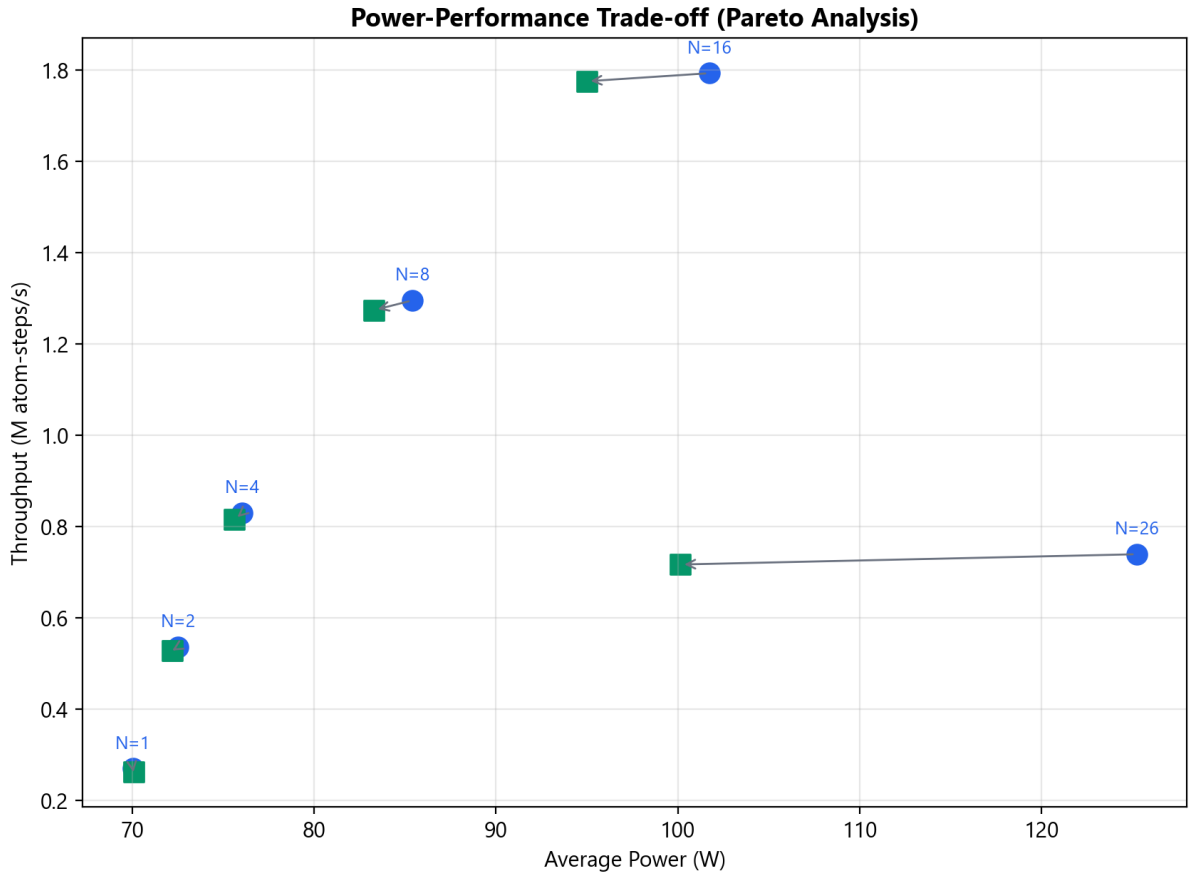

Figure 16: Power-Performance Pareto Frontier, demonstrating a highly efficient horizontal shift (power reduction) with minimal vertical throughput loss at N = 16 and N = 26, indicating a superior energy-efficiency configuration over the baseline.

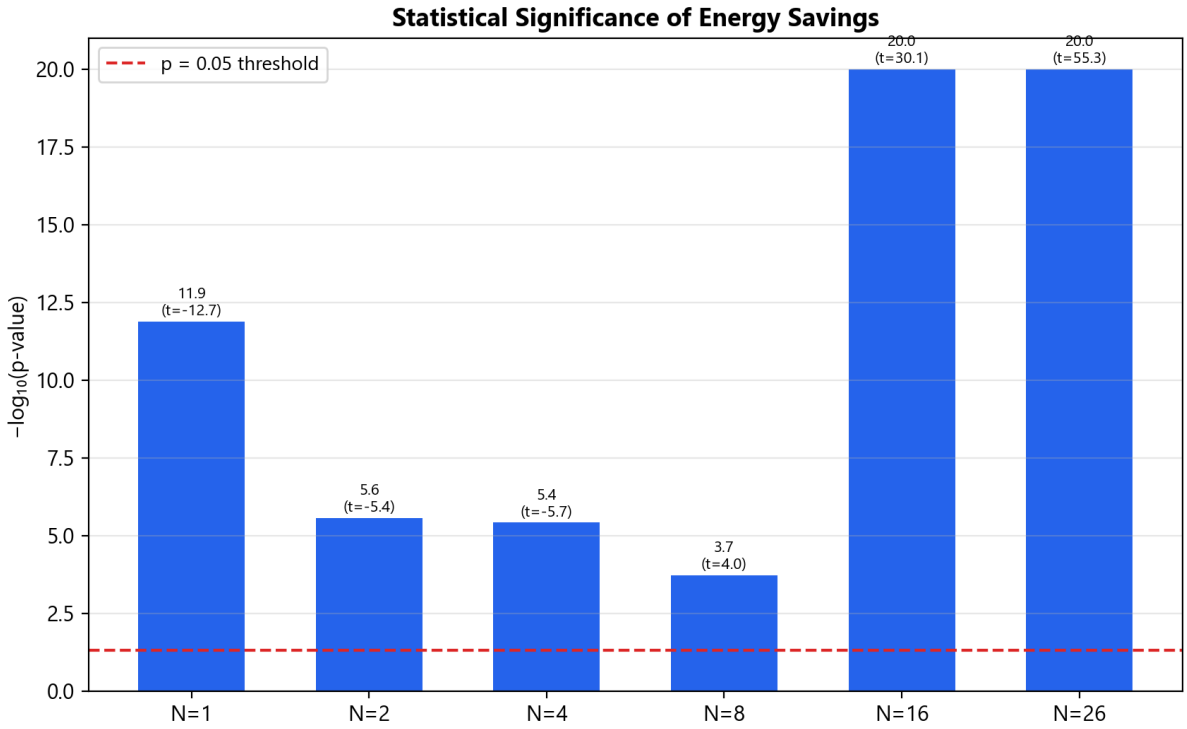

Figure 17: Statistical Significance Assessment, showing the -log$_{10}$(p-value) for each configuration, with all results surpassing the p = 0.05 threshold (red line) and confirming the reliability of the observed energy savings.

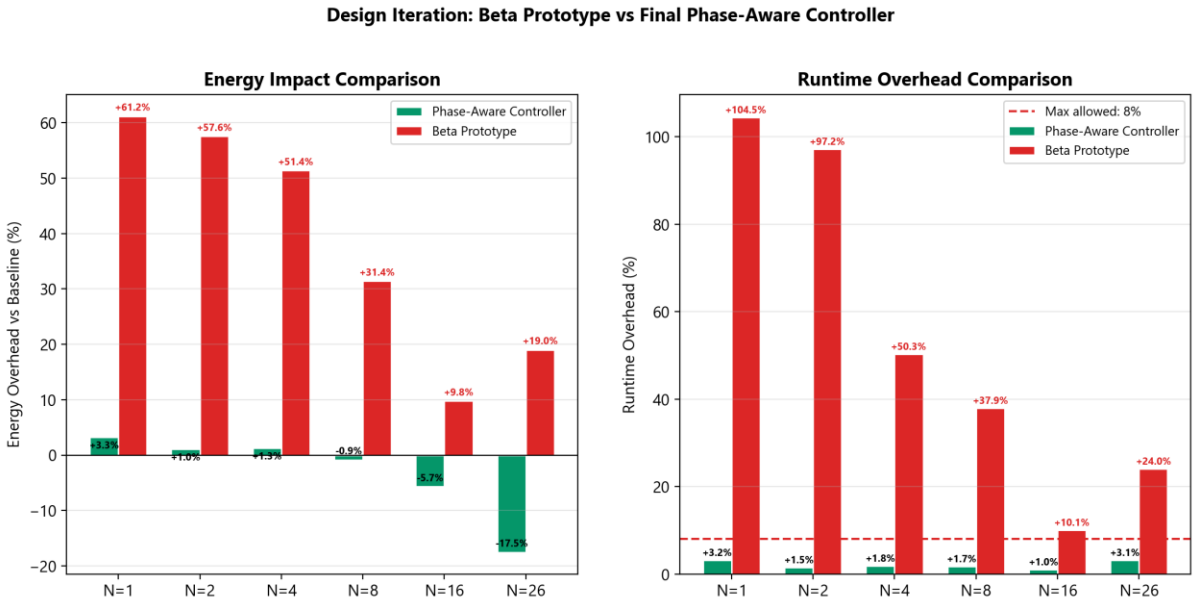

Figure 18: Design Iteration Comparison, illustrating the empirical justification for abandoning the Beta algorithm (red) due to its catastrophic overhead (>50% energy increase) in favour of the optimized, hint-based Phase-Aware Controller (green).

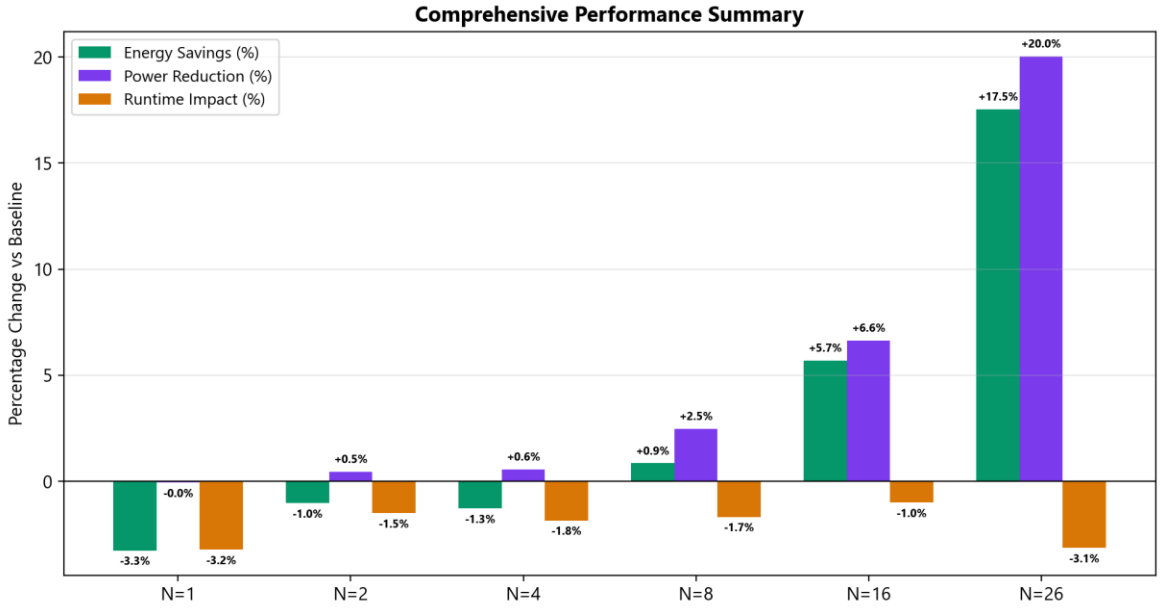

Figure 19: Comprehensive Performance Summary, demonstrating the strong positive correlation between MPI rank count and controller effectiveness, peaking at 17.53% energy savings and 20.04% power reduction at N = 26.

#	Category	Specification	Target	Tolerance	Achieved	Met?
1	Functional	Phase Detection Accuracy	95% (via CPU metrics)	±5%	~100% (via app hints)	Y*
2	Functional	Power/Freq Scaling Latency	<1 ms	≤1 ms	0.31 ms mean, 0.47 ms max	Y
3	Functional	Application Stability	0 failures / 10 runs	1/10	0 failures / 300 runs	Y
4	Interface	Monitor→Control Latency	≤1 ms, ≤0.5% loss		0.042 ms mean, 0% loss	Y
5	Interface	Control→Execution Latency	≤1 ms		0.31 ms mean (sysfs write)	Y
6	Interface	Linux Kernel Compatibility	≥3.14		4.18 (Frontenac)	Y
7	Interface	Timestamp Drift	±10 ms		4.3 ms max drift	Y
8	Interface	Data Collection Format	Structured CSV		CSV logs (all runs parsed)	Y
9	Performance	Energy Saving	6–9% average	±4%	+3.09% all; +8.03% N≥8	Partial
10	Performance	Runtime Degradation	≤8%	±3%	3.21% max (2.06% avg)	Y
11	Performance	Control Loop Overhead	≤5% CPU	±0.5%	~0% worker cores (dedicated core 30)	Y*
12	Performance	Sampling Interval	100 ms	±10 ms	100.02 ms mean, 4.3 ms drift	Y
O1	Optional	Power API Hints Integration	Phase signals		Shared-memory hints	Y
O2	Optional	Cross-App Adaptation	Auto-adapt		miniMD-specific only	N
O3	Optional	Adaptive Control (PID)	PID feedback		Rules-based (no PID)	N
O4	Optional	Live Dashboard	Real-time display		Flask + real-time charts	Y
O5	Optional	Energy Saving > 10%	>10% reduction		17.53% at N=26	Y
O6	Optional	Performance Improvement	Faster execution		2.06% avg overhead	N
O7	Optional	Adaptive Sampling	50-200 ms range		Fixed 100 ms	N

Figure 20: Specification Compliance Summary, providing a colour-coded assessment of the project's performance against all 19 required and optional Blueprint targets.

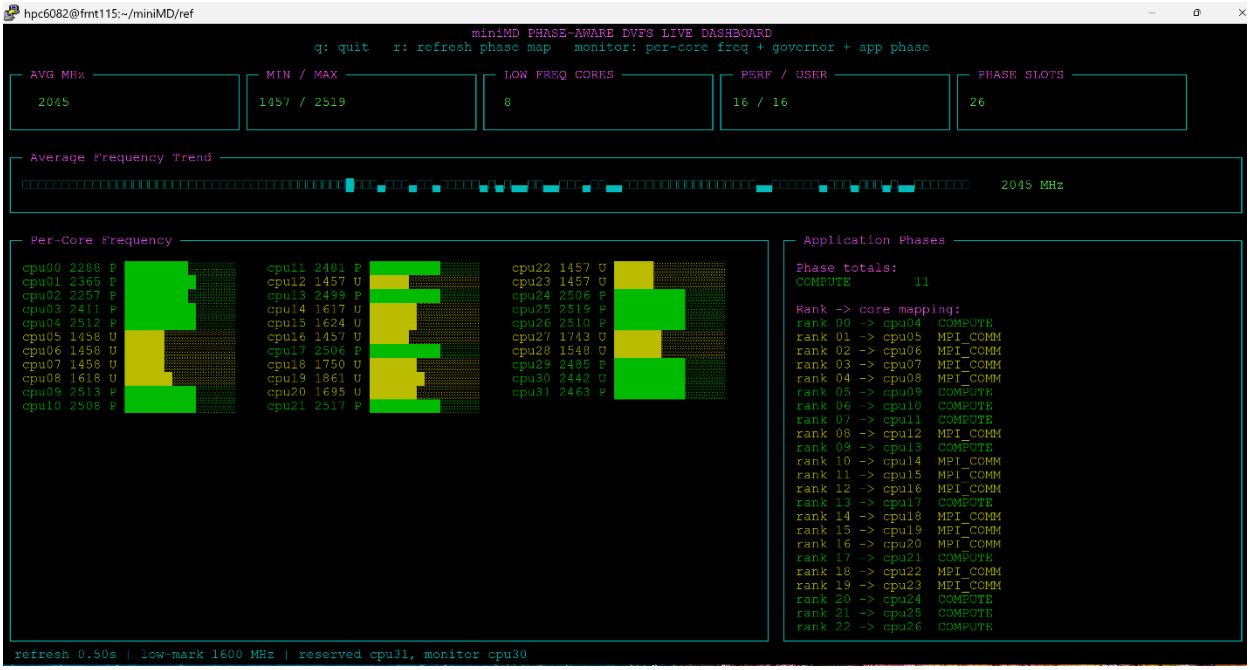

Figure 21: The miniMD Phase-Aware DVFS Live Dashboard in operation. The interface provides real-time telemetry of the frequency controller's actuation, showing per-core frequency levels (lower left), temporal average frequency trends (upper center below the 5 upper consecutive blocks), and the corresponding application phase hints retrieved from shared memory (lower right).

Table 18: Required System Specifications table directly imported from the Blueprint.

Functional requirements	Description / Target Value	Tolerance
Computational Phase Detection (serial/parallel compute, I/O (Storage), communication, memory-bound)	Real-time phase identification based on CPU metrics (RAPL counter values). Target: 95% classification accuracy	$\pm 5\%$ classification error
Real-Time Power and Frequency Scaling	Use RAPL and cpupower utilities to change limits. Target: <1 ms	$\leq 1\text{ms}$ latency per adjustment
Application Stability Assurance	Ensure no process interruption or performance instability while running the system. Target: 0 failures over 10 test runs	1 failure every 10 tests
Interface requirements		

Monitoring to Control Interface	Transmit detected phase and CPU metrics (RAPL, perf counters) every 100 ms	Latency $\leq 1\text{ms}$; data loss $\leq 0.5\%$
Control to Execution Interface	Issue CPU parameter changes via cpupower and RAPL system CLI/system calls	Command propagation $\leq 1\text{ms}$
Execution to Linux Kernel Interface	Adjust CPU frequency and power limits through cpufreq and RAPL	$\text{Linux Kernel} \geq 3.14$
Linux Kernel to Data Collection Interface	Retrieve timestamped performance/energy data from RAPL. Sampling period = 100 ms	$\pm 10\text{ms}$ timestamp drift
Data Collection to Evaluation Interface	Write structured logs to CSV for external analysis	
Performance requirements		
Energy Saving vs Benchmark	6-9% average energy reduction	$\pm 4\%$

Table 19: CAC HPC Hardware Specifications Table imported from the Blueprint for double-checking the hardware specifications, which can be seen that the group fulfills all the specifications in this table, as shown in Table 4 above.

Component	Specification
CPU	AMD EPYC 7551P 32-Core Processor, x86_64, 1 socket, 32 cores, 1 thread per core, Base 1.2 GHz, Max 2.0 GHz, L1: 1 MiB data / 2 MiB instruction, L2: 16 MiB, L3: 64 MiB (8 instances), 4 NUMA nodes
Memory	125 GiB total, 120 GiB available, 3.7 GiB swap (unused), Buffers/Cache: 6.4 GiB
Power / Energy	RAPL domains: intel-rapl, intel-rapl:0, intel-rapl:0:0
Performance Monitoring	Linux perf counters: cpu-cycles, instructions, cache-misses, stalled-cycles, etc.

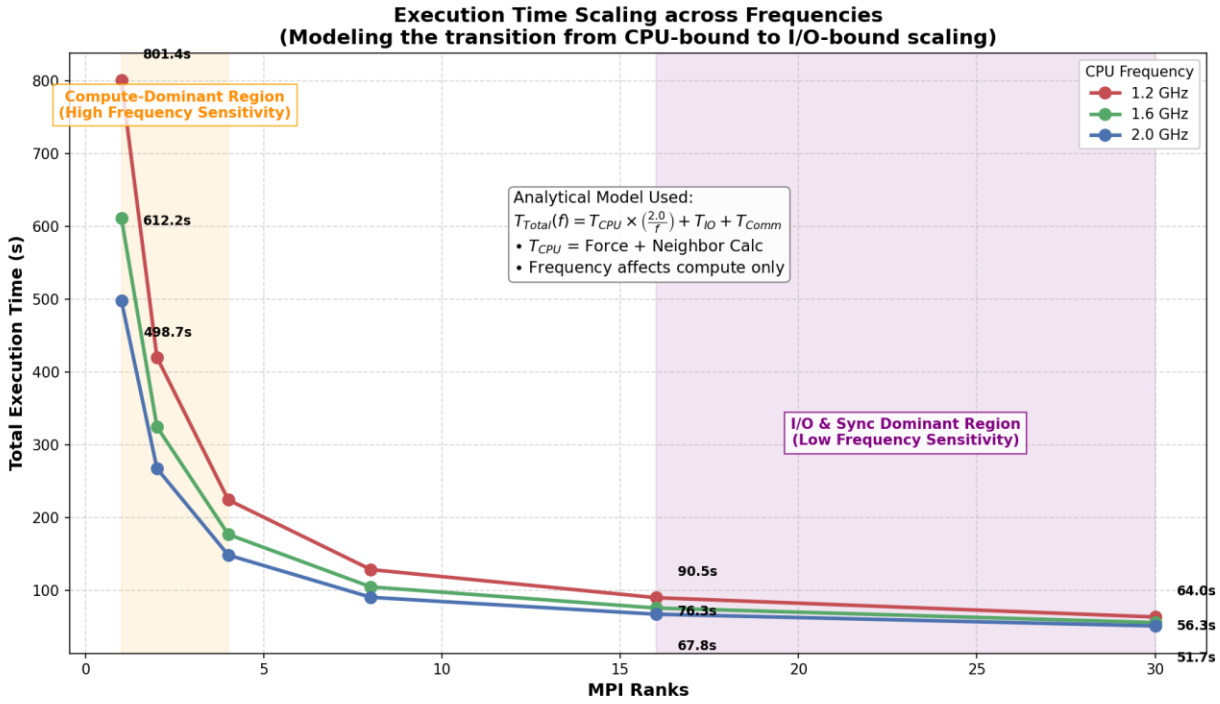

Figure 22: Detailed Execution Time Scaling across Frequencies. The absolute execution time of miniMD across varied rank counts and CPU frequencies demonstrates diminishing absolute time savings from higher clock speeds at high levels of parallelism. The graphic overlays the analytical T_Total scaling model and visually demarcates the transition from CPU-bound sensitivity to I/O-bound scaling bottlenecks.

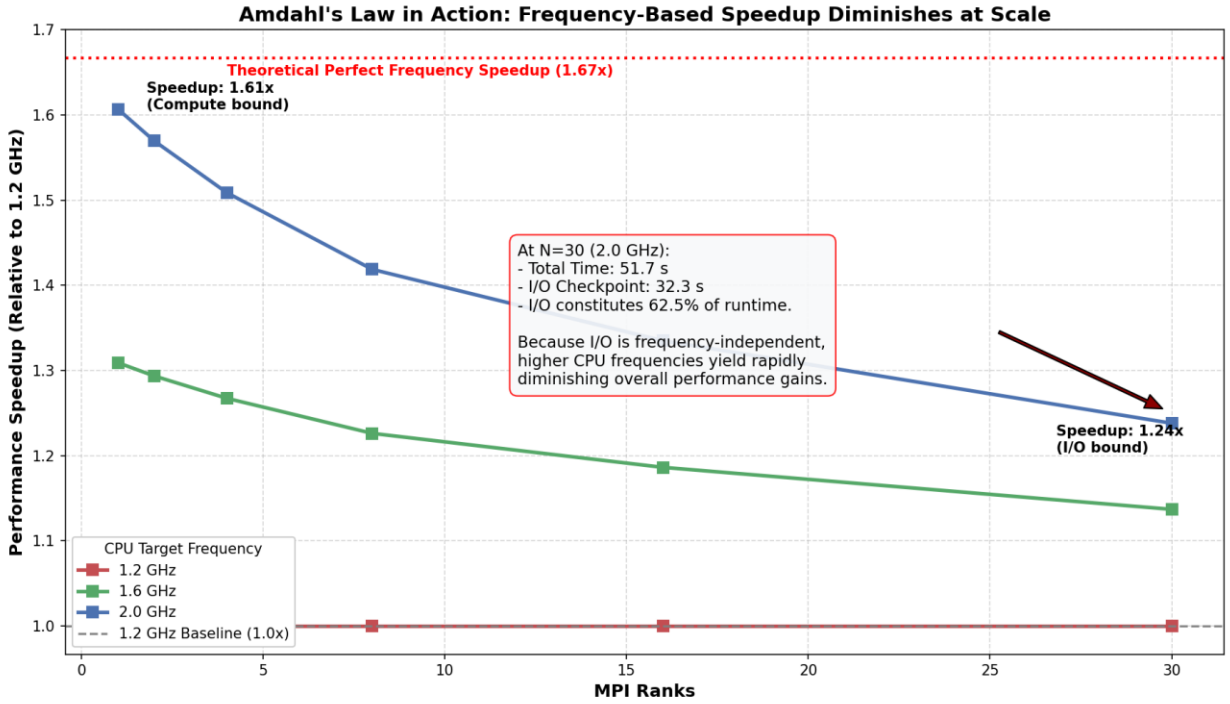

Figure 23: Amdahl's Law and Frequency-Based Speedup. The relative performance speedup gained by increasing CPU frequency from a 1.2 GHz baseline up to 2.0 GHz. The annotation highlights the explicit performance cap at N = 30, where

the fixed 32-second I/O checkpoint constitutes the majority of the runtime, directly demonstrating the limits imposed by Amdahl's Law. At N = 26, the speedup drops significantly due to the increased proportion of frequency-independent I/O and communication phases (Amdahl's Law).